

Rainbow: Combining Improvements in Deep Reinforcement Learning

Matteo Hessel
DeepMind

Joseph Modayil
DeepMind

Hado van Hasselt
DeepMind

Tom Schaul
DeepMind

Georg Ostrovski
DeepMind

Will Dabney
DeepMind

Dan Horgan
DeepMind

Bilal Piot
DeepMind

Mohammad Azar
DeepMind

David Silver
DeepMind

Abstract

The deep reinforcement learning community has made several independent improvements to the DQN algorithm. However, it is unclear which of these extensions are complementary and can be fruitfully combined. This paper examines six extensions to the DQN algorithm and empirically studies their combination. Our experiments show that the combination provides state-of-the-art performance on the Atari 2600 benchmark, both in terms of data efficiency and final performance. We also provide results from a detailed ablation study that shows the contribution of each component to overall performance.

Introduction

The many recent successes in scaling reinforcement learning (RL) to complex sequential decision-making problems were kick-started by the Deep Q-Networks algorithm (DQN; Mnih et al. 2013, 2015). Its combination of Q-learning with convolutional neural networks and experience replay enabled it to learn, from raw pixels, how to play many Atari games at human-level performance. Since then, many extensions have been proposed that enhance its speed or stability.

Double DQN (DDQN; van Hasselt, Guez, and Silver 2016) addresses an overestimation bias of Q-learning (van Hasselt 2010), by decoupling selection and evaluation of the bootstrap action. Prioritized experience replay (Schaul et al. 2015) improves data efficiency, by replaying more often transitions from which there is more to learn. The dueling network architecture (Wang et al. 2016) helps to generalize across actions by separately representing state values and action advantages. Learning from multi-step bootstrap targets (Sutton 1988; Sutton and Barto 1998), as used in A3C (Mnih et al. 2016), shifts the bias-variance trade-off and helps to propagate newly observed rewards faster to earlier visited states. Distributional Q-learning (Bellemare, Dabney, and Munos 2017) learns a categorical distribution of discounted returns, instead of estimating the mean. Noisy DQN (Fortunato et al. 2017) uses stochastic network layers for exploration. This list is, of course, far from exhaustive.

Each of these algorithms enables substantial performance improvements in isolation. Since they do so by addressing

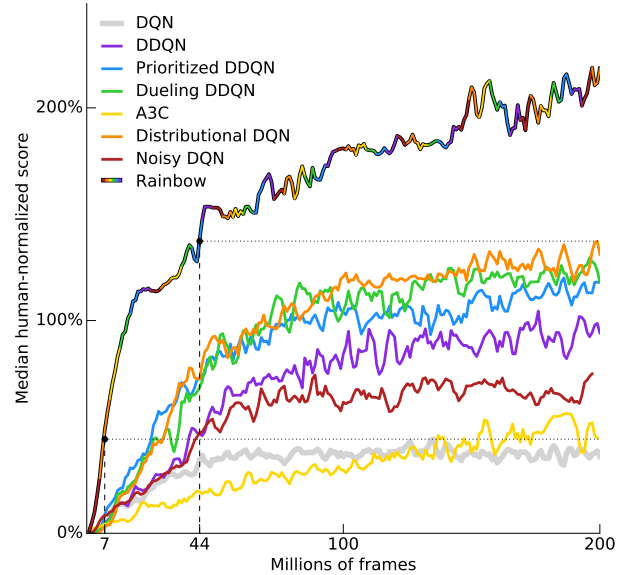


Figure 1: **Median human-normalized performance** across 57 Atari games. We compare our integrated agent (rainbow-colored) to DQN (grey) and six published baselines. Note that we match DQN’s best performance after 7M frames, surpass any baseline within 44M frames, and reach substantially improved final performance. Curves are smoothed with a moving average over 5 points.

radically different issues, and since they build on a shared framework, they could plausibly be combined. In some cases this has been done: Prioritized DDQN and Dueling DDQN both use double Q-learning, and Dueling DDQN was also combined with prioritized experience replay. In this paper we propose to study an agent that combines all the aforementioned ingredients. We show how these different ideas can be integrated, and that they are indeed largely complementary. In fact, their combination results in new state-of-the-art results on the benchmark suite of 57 Atari 2600 games from the Arcade Learning Environment (Bellemare et al. 2013), both in terms of data efficiency and of final performance. Finally we show results from ablation studies to help understand the contributions of the different components.

Rainbow: 结合深度强化学习的改进

Matteo Hessel
DeepMind

Joseph Modayil
DeepMind

Hado van Hasselt
DeepMind

Tom Schaul
DeepMind

Georg Ostrovski
DeepMind

Will Dabney
DeepMind

Dan Horgan
DeepMind

Bilal Piot
DeepMind

Mohammad Azar
DeepMind

David Silver
DeepMind

抽象的

深度强化学习社区对 DQN 算法进行了多项独立改进。然而，尚不清楚这些扩展中哪些是互补的并且可以有效地组合。本文研究了 DQN 算法的六种扩展，并实证研究了它们的组合。我们的实验表明，该组合在 Atari 2600 基准测试中提供了最先进的性能，无论是在数据效率还是最终性能方面。我们还提供了详细的消融研究的结果，显示了每个组件对整体性能的贡献。

介绍

最近在将强化学习 (RL) 扩展到复杂的顺序决策问题方面取得的许多成功都是由深度 Q 网络算法启动的 (DQN; Mnih 等人, 2013 年, 2015 年)。它将 Q 学习与卷积神经网络和经验回放相结合，使其能够从原始像素中学习如何以人类水平的表现玩许多 Atari 游戏。从那时起，人们提出了许多扩展来提高其速度或稳定性。

Double DQN (DDQN; van Hasselt, Guez 和 Silver 2016) 通过解耦引导操作的选择和评估来解决 Q 学习的高估偏差 (van Hasselt 2010)。优先经验回放 (Schaul et al. 2015) 通过更频繁地重放十个转换来提高数据效率，从中可以学到更多东西。决斗网络架构 (Wang et al. 2016) 通过分别表示状态值和动作优势来帮助泛化动作。从多步引导目标中学习 (Sutton 1988; Sutton 和 Barto 1998)，如 A3C 中所使用的 (Mnih 等人, 2016)，改变了偏差-方差权衡，并有助于将新观察到的奖励更快地传播到早期访问的状态。分布式 Q 学习 (Bellemare, Dabney 和 Munos 2017) 学习贴现收益的分类分布，而不是估计平均值。Noisy DQN (Fortunato et al. 2017) 使用随机网络层进行探索。当然，这个清单远非详尽无遗。

这些算法中的每一种都可以单独实现显著的性能改进。因为他们这样做是通过解决

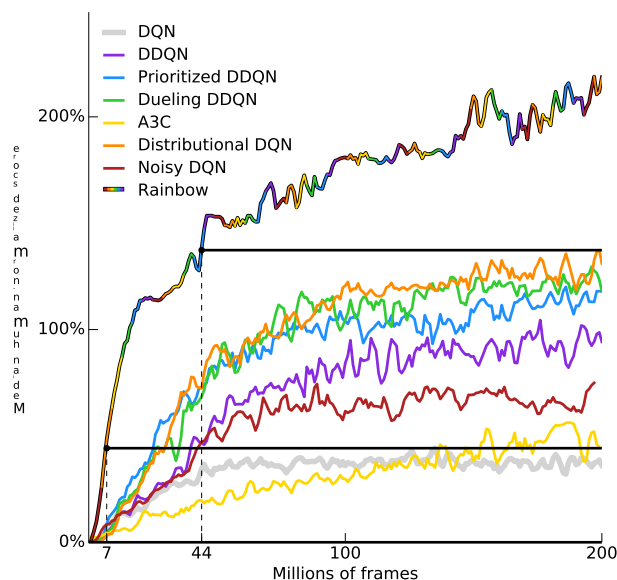


图 1: 57 款 Atari 游戏的人类标准化表现中位数。我们将我们的集成代理 (彩虹色) 与 DQN (灰色) 和六个已发布的基线进行比较。请注意，我们在 7M 帧后匹配 DQN 的最佳性能，超过 44M 帧内的任何基线，并达到显著提高的最终性能。曲线使用超过 5 个点的移动平均值进行平滑。

完全不同的问题，由于它们建立在一个共享的框架之上，因此它们可以合理地结合起来。在某些情况下已经这样做了：优先 DDQN 和决斗 DDQN 都使用双 Q 学习，并且决斗 DDQN 还与优先经验回放相结合。在本文中，我们建议研究一种结合了所有上述成分的制剂。我们展示了如何整合这些不同的想法，并且它们确实在很大程度上是互补的。事实上，它们的结合在 Arcade 学习环境 (Bellemare 等人, 2013) 的 57 个 Atari 2600 游戏的基准套件上产生了新的最先进的结果，无论是在数据效率还是最终性能方面。最后，我们展示了消融研究的结果，以帮助了解不同成分的贡献。

Background

Reinforcement learning addresses the problem of an *agent* learning to act in an *environment* in order to maximize a scalar *reward* signal. No direct supervision is provided to the agent, for instance it is never directly told the best action.

Agents and environments. At each discrete time step $t = 0, 1, 2, \dots$, the environment provides the agent with an observation S_t , the agent responds by selecting an action A_t , and then the environment provides the next reward R_{t+1} , discount γ_{t+1} , and state S_{t+1} . This interaction is formalized as a *Markov Decision Process*, or MDP, which is a tuple $\langle \mathcal{S}, \mathcal{A}, T, r, \gamma \rangle$, where $\mathcal{S}$ is a finite set of states, $\mathcal{A}$ is a finite set of actions, $T(s, a, s') = P[S_{t+1} = s' \mid S_t = s, A_t = a]$ is the (stochastic) transition function, $r(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$ is the reward function, and $\gamma \in [0, 1]$ is a discount factor. In our experiments MDPs will be *episodic* with a constant $\gamma_t = \gamma$, except on episode termination where $\gamma_t = 0$, but the algorithms are expressed in the general form.

On the agent side, action selection is given by a policy π that defines a probability distribution over actions for each state. From the state S_t encountered at time t , we define the discounted return $G_t = \sum_{k=0}^{\infty} \gamma_t^{(k)} R_{t+k+1}$ as the discounted sum of future rewards collected by the agent, where the discount for a reward k steps in the future is given by the product of discounts before that time, $\gamma_t^{(k)} = \prod_{i=1}^k \gamma_{t+i}$. An agent aims to maximize the expected discounted return by finding a good policy.

The policy may be learned directly, or it may be constructed as a function of some other learned quantities. In value-based reinforcement learning, the agent learns an estimate of the expected discounted return, or value, when following a policy π starting from a given state, $v^\pi(s) = E_\pi[G_t \mid S_t = s]$, or state-action pair, $q^\pi(s, a) = E_\pi[G_t \mid S_t = s, A_t = a]$. A common way of deriving a new policy from a state-action value function is to act ϵ -greedily with respect to the action values. This corresponds to taking the action with the highest value (the *greedy* action) with probability $(1 - \epsilon)$, and to otherwise act uniformly at random with probability ϵ . Policies of this kind are used to introduce a form of *exploration*: by randomly selecting actions that are sub-optimal according to its current estimates, the agent can discover and correct its estimates when appropriate. The main limitation is that it is difficult to discover alternative courses of action that extend far into the future; this has motivated research on more directed forms of exploration.

Deep reinforcement learning and DQN. Large state and/or action spaces make it intractable to learn Q value estimates for each state and action pair independently. In deep reinforcement learning, we represent the various components of agents, such as policies $\pi(s, a)$ or values $q(s, a)$, with deep (i.e., multi-layer) neural networks. The parameters of these networks are trained by gradient descent to minimize some suitable loss function.

In DQN (Mnih et al. 2015) deep networks and reinforcement learning were successfully combined by using a convolutional neural net to approximate the action values for a

given state S_t (which is fed as input to the network in the form of a stack of raw pixel frames). At each step, based on the current state, the agent selects an action ϵ -greedily with respect to the action values, and adds a transition $(S_t, A_t, R_{t+1}, \gamma_{t+1}, S_{t+1})$ to a replay memory buffer (Lin 1992), that holds the last million transitions. The parameters of the neural network are optimized by using stochastic gradient descent to minimize the loss

$$(R_{t+1} + \gamma_{t+1} \max_{a'} q_{\bar{\theta}}(S_{t+1}, a') - q_{\theta}(S_t, A_t))^2, \quad (1)$$

where t is a time step randomly picked from the replay memory. The gradient of the loss is back-propagated only into the parameters θ of the *online network* (which is also used to select actions); the term $\bar{\theta}$ represents the parameters of a *target network*; a periodic copy of the online network which is not directly optimized. The optimization is performed using RMSprop (Tieleman and Hinton 2012), a variant of stochastic gradient descent, on mini-batches sampled uniformly from the experience replay. This means that in the loss above, the time index t will be a random time index from the last million transitions, rather than the current time. The use of experience replay and target networks enables relatively stable learning of Q values, and led to super-human performance on several Atari games.

Extensions to DQN

DQN has been an important milestone, but several limitations of this algorithm are now known, and many extensions have been proposed. We propose a selection of six extensions that each have addressed a limitation and improved overall performance. To keep the size of the selection manageable, we picked a set of extensions that address distinct concerns (e.g., just one of the many addressing exploration).

Double Q-learning. Conventional Q-learning is affected by an overestimation bias, due to the maximization step in Equation 1, and this can harm learning. Double Q-learning (van Hasselt 2010), addresses this overestimation by decoupling, in the maximization performed for the bootstrap target, the selection of the action from its evaluation. It is possible to effectively combine this with DQN (van Hasselt, Guez, and Silver 2016), using the loss

$$(R_{t+1} + \gamma_{t+1} q_{\bar{\theta}}(S_{t+1}, \arg\max_{a'} q_{\theta}(S_{t+1}, a')) - q_{\theta}(S_t, A_t))^2.$$

This change was shown to reduce harmful overestimations that were present for DQN, thereby improving performance.

Prioritized replay. DQN samples uniformly from the replay buffer. Ideally, we want to sample more frequently those transitions from which there is much to learn. As a proxy for learning potential, prioritized experience replay (Schaul et al. 2015) samples transitions with probability p_t relative to the last encountered absolute *TD error*:

$$p_t \propto \left| R_{t+1} + \gamma_{t+1} \max_{a'} q_{\bar{\theta}}(S_{t+1}, a') - q_{\theta}(S_t, A_t) \right|^\omega,$$

where ω is a hyper-parameter that determines the shape of the distribution. New transitions are inserted into the replay

背景

强化学习解决了 *agent* 学习在 *environment* 中采取行动以最大化标量 *reward* 信号的问题。没有向代理提供直接监督，例如，它永远不会直接被告知最佳行动。

代理和环境。在每个离散时间步 $t = 0, 1, 2 \dots$ ，环境为智能体提供观察 S_t ，智能体通过选择动作 A_t 进行响应，然后环境提供下一个奖励 R_{t+1} 、折扣 γ_{t+1} 和状态 S_{t+1} 。这种交互被形式化为 *Markov Decision Process* 或 MDP，它是一个元组 $\langle \mathcal{S}, \mathcal{A}, T, r, \gamma \rangle$ ，其中 $\mathcal{S}$ 是有限状态集， $\mathcal{A}$ 是有限动作集， $T(s, a, s') = P[S_{t+1} = s' | S_t = s, A_t = a]$ 是（随机）转换函数， $r(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a]$ 是奖励函数， $\gamma \in [0, 1]$ 是折扣因子。在我们的实验中，MDP 将是具有常量 $\gamma_t = \gamma$ 的 *episodic*，除了在情节终止时 $\gamma_t = 0$ ，但算法以一般形式表示。

在代理方面，动作选择由策略 π 给出，该策略定义每个状态的动作概率分布。根据在时间 t 遇到的状态 S_t ，我们将折扣回报 $G_t = \sum_{k=0}^{\infty} \gamma_t^{(k)} R_{t+k+1}$ 定义为代理收集的将来奖励的折扣总和，其中未来奖励 k 步的折扣由该时间之前的折扣的乘积 $\gamma_t^{(k)} = \prod_{i=1}^k \gamma_{t+i}$ 给出。代理人的目标是通过找到一个好的政策来最大化预期的折扣回报。

该策略可以直接学习，也可以构建为一些其他学习量的函数。在基于价值的强化学习中，当从给定状态 $v^\pi(s) = E_\pi[G_t | S_t = s]$ 或状态-动作对 $q^\pi(s, a) = E_\pi[G_t | S_t = s, A_t = a]$ 开始遵循策略 π 时，代理学习预期折扣回报或价值的估计。从状态-动作值函数导出新策略的常见方法是对动作值采取 ϵ 贪婪行为。这对应于以概率 $(1-\epsilon)$ 采取具有最高值的动作（*greedy* 动作），否则以概率 ϵ 均匀随机动作。此类策略用于引入一种形式的 *explore-ration*：通过根据当前估计随机选择次优动作，智能体可以在适当的时候发现并纠正其估计。主要的限制是很难发现可以延续到很远的未来的替代行动方案。这激发了对更有针对性的探索形式的研究。

深度强化学习和 DQN。较大的状态和/或动作空间使得独立学习每个状态和动作对的 Q 值估计变得困难。在深度强化学习中，我们用深度（即多层）神经网络表示代理的各个组成部分，例如策略 $\pi(s, a)$ 或值 $q(s, a)$ 。这些网络的参数通过梯度下降进行训练，以最小化一些合适的损失函数。

在 DQN (Mnih 等人, 2015) 中，通过使用卷积神经网络来近似一个动作值，成功地将深度网络和强化学习结合起来。

给定状态 S_t （它以原始像素帧堆栈）的形式作为网络的输入。在每一步中，代理根据当前状态贪婪地选择一个动作 ϵ ，并将一个转换 $(S_t, A_t, R_{t+1}, \gamma_{t+1}, S_{t+1})$ 添加到重放内存缓冲区 (Lin 1992)，该缓冲区保存最后一百万个转换。通过使用随机梯度下降来优化神经网络的参数以最小化损失

$$(R_{t+1} + \gamma_{t+1} \max_{a'} q_{\bar{\theta}}(S_{t+1}, a') - q_{\theta}(S_t, A_t))^2, \quad (1)$$

其中 t 是从重播内存中随机选取的时间步。损失的梯度仅反向传播到 *online network*（的参数 θ 中，该参数也用于选择动作）；项 θ 代表 *target network* 的参数；未直接优化的在线网络的定期副本。使用 RMSprop (Tieleman 和 Hinton 2012)（随机梯度下降的一种变体）对从经验回放中均匀采样的小批量进行优化。这意味着在上面的损失中，时间索引 t 将是最后一百万个转换的随机时间索引，而不是当前时间。使用经验回放和目标网络可以实现相对稳定的 Q 值学习，并在一些 Atari 游戏上实现了超人的表现。

DQN 的扩展

DQN 是一个重要的里程碑，但现在人们已经知道该算法的一些局限性，并且已经提出了许多扩展。我们建议选择六个扩展，每个扩展都解决了限制并提高了整体性能。为了保持选择的大小易于管理，我们选择了一组解决不同问题的扩展（例如，只是众多解决探索问题中的一个）。

双Q学习。由于等式 1 中的最大化步骤，传统的 Q 学习会受到高估偏差的影响，这可能会损害学习。双 Q 学习 (van Hasselt 2010) 通过解耦解决了这种高估问题，在为引导目标执行的最大化中，从其评估中选择动作。可以使用损失函数将其与 DQN 有效结合 (van Hasselt、Guez 和 Silver 2016)

$$(R_{t+1} + \gamma_{t+1} q_{\bar{\theta}}(S_{t+1}, \arg\max_{a'} q_{\theta}(S_{t+1}, a')) - q_{\theta}(S_t, A_t))^2.$$

这一变化被证明可以减少 DQN 中存在的有害高估，从而提高性能。

优先重播。DQN 从重播缓冲区中统一采样。理想情况下，我们希望更频繁地对那些有很多值得学习的转变进行采样。作为学习潜力的代理，优先经验重播 (Schaul et al. 2015) 以相对于上次遇到的绝对 *TD error* 的概率 p_t 对转换进行采样：

$$p_t \propto \left| R_{t+1} + \gamma_{t+1} \max_{a'} q_{\bar{\theta}}(S_{t+1}, a') - q_{\theta}(S_t, A_t) \right|^\omega,$$

其中 ω 是确定分布形状的超参数。新的转场被插入到重播中

buffer with maximum priority, providing a bias towards recent transitions. Note that stochastic transitions might also be favoured, even when there is little left to learn about them.

Dueling networks. The dueling network is a neural network architecture designed for value based RL. It features two streams of computation, the value and advantage streams, sharing a convolutional encoder, and merged by a special aggregator (Wang et al. 2016). This corresponds to the following factorization of action values:

$$q_\theta(s, a) = v_\eta(f_\xi(s)) + a_\psi(f_\xi(s), a) - \frac{\sum_{a'} a_\psi(f_\xi(s), a')}{N_{\text{actions}}},$$

where ξ , η , and ψ are, respectively, the parameters of the shared encoder f_ξ , of the value stream v_η , and of the advantage stream a_ψ ; and $\theta = \{\xi, \eta, \psi\}$ is their concatenation.

Multi-step learning. Q-learning accumulates a single reward and then uses the greedy action at the next step to bootstrap. Alternatively, forward-view *multi-step* targets can be used (Sutton 1988). We define the truncated n -step return from a given state S_t as

$$R_t^{(n)} \equiv \sum_{k=0}^{n-1} \gamma_t^{(k)} R_{t+k+1}. \quad (2)$$

A multi-step variant of DQN is then defined by minimizing the alternative loss,

$$(R_t^{(n)} + \gamma_t^{(n)} \max_{a'} q_\theta(S_{t+n}, a') - q_\theta(S_t, A_t))^2.$$

Multi-step targets with suitably tuned n often lead to faster learning (Sutton and Barto 1998).

Distributional RL. We can learn to approximate the distribution of returns instead of the expected return. Recently Bellemare, Dabney, and Munos (2017) proposed to model such distributions with probability masses placed on a discrete support $\mathbf{z}$, where $\mathbf{z}$ is a vector with $N_{\text{atoms}} \in \mathbb{N}^+$ atoms, defined by $z^i = v_{\min} + (i - 1) \frac{v_{\max} - v_{\min}}{N_{\text{atoms}} - 1}$ for $i \in \{1, \dots, N_{\text{atoms}}\}$. The approximating distribution d_t at time t is defined on this support, with the probability mass $p_\theta^i(S_t, A_t)$ on each atom i , such that $d_t = (\mathbf{z}, \mathbf{p}_\theta(S_t, A_t))$. The goal is to update θ such that this distribution closely matches the actual distribution of returns.

To learn the probability masses, the key insight is that return distributions satisfy a variant of Bellman’s equation. For a given state S_t and action A_t , the distribution of the returns under the optimal policy π^* should match a target distribution defined by taking the distribution for the next state S_{t+1} and action $a_{t+1}^* = \pi^*(S_{t+1})$, contracting it towards zero according to the discount, and shifting it by the reward (or distribution of rewards, in the stochastic case). A distributional variant of Q-learning is then derived by first constructing a new support for the target distribution, and then minimizing the Kullback-Leibler divergence between the distribution d_t and the target distribution $d'_t \equiv (R_{t+1} + \gamma_{t+1} \mathbf{z}, \mathbf{p}_{\bar{\theta}}(S_{t+1}, a_{t+1}^*))$,

$$D_{\text{KL}}(\Phi_{\mathbf{z}} d'_t || d_t). \quad (3)$$

Here $\Phi_{\mathbf{z}}$ is a L2-projection of the target distribution onto the fixed support $\mathbf{z}$, and $\bar{a}_{t+1}^* = \text{argmax}_a q_{\bar{\theta}}(S_{t+1}, a)$ is the greedy action with respect to the mean action values $q_{\bar{\theta}}(S_{t+1}, a) = \mathbf{z}^\top \mathbf{p}_{\bar{\theta}}(S_{t+1}, a)$ in state S_{t+1} .

As in the non-distributional case, we can use a frozen copy of the parameters $\bar{\theta}$ to construct the target distribution. The parametrized distribution can be represented by a neural network, as in DQN, but with $N_{\text{atoms}} \times N_{\text{actions}}$ outputs. A *softmax* is applied independently for each action dimension of the output to ensure that the distribution for each action is appropriately normalized.

Noisy Nets. The limitations of exploring using ϵ -greedy policies are clear in games such as Montezuma’s Revenge, where many actions must be executed to collect the first reward. Noisy Nets (Fortunato et al. 2017) propose a noisy linear layer that combines a deterministic and noisy stream,

$$\mathbf{y} = (\mathbf{b} + \mathbf{W}\mathbf{x}) + (\mathbf{b}_{\text{noisy}} \odot \epsilon^b + (\mathbf{W}_{\text{noisy}} \odot \epsilon^w) \mathbf{x}), \quad (4)$$

where ϵ^b and ϵ^w are random variables, and $\odot$ denotes the element-wise product. This transformation can then be used in place of the standard linear $\mathbf{y} = \mathbf{b} + \mathbf{W}\mathbf{x}$. Over time, the network can learn to ignore the noisy stream, but will do so at different rates in different parts of the state space, allowing state-conditional exploration with a form of self-annealing.

The Integrated Agent

In this paper we integrate all the aforementioned components into a single integrated agent, which we call *Rainbow*.

First, we replace the 1-step distributional loss (3) with a multi-step variant. We construct the target distribution by contracting the value distribution in S_{t+n} according to the cumulative discount, and shifting it by the truncated n -step discounted return. This corresponds to defining the target distribution as $d_t^{(n)} = (R_t^{(n)} + \gamma_t^{(n)} \mathbf{z}, \mathbf{p}_{\bar{\theta}}(S_{t+n}, a_{t+n}^*))$. The resulting loss is

$$D_{\text{KL}}(\Phi_{\mathbf{z}} d_t^{(n)} || d_t),$$

where, again, $\Phi_{\mathbf{z}}$ is the projection onto $\mathbf{z}$.

We combine the multi-step distributional loss with double Q-learning by using the greedy action in S_{t+n} selected according to the *online network* as the bootstrap action a_{t+n}^* , and evaluating such action using the *target network*.

In standard proportional prioritized replay (Schaul et al. 2015) the absolute TD error is used to prioritize the transitions. This can be computed in the distributional setting, using the mean action values. However, in our experiments all distributional Rainbow variants prioritize transitions by the KL loss, since this is what the algorithm is minimizing:

$$p_t \propto \left(D_{\text{KL}}(\Phi_{\mathbf{z}} d_t^{(n)} || d_t) \right)^\omega.$$

The KL loss as priority might be more robust to noisy stochastic environments because the loss can continue to decrease even when the returns are not deterministic.

The network architecture is a dueling network architecture adapted for use with return distributions. The network

具有最高优先级的缓冲区，提供对最近转换的偏向。请注意，随机转变也可能受到青睐，即使几乎没有什么可以了解它们。

决斗网络。决斗网络是一种专为基于价值的强化学习而设计的神经网络架构。它具有两个计算流，即价值流和优势流，共享一个卷积编码器，并由一个特殊的聚合器合并（Wang et al. 2016）。这对应于以下动作值因式分解：

$$q_\theta(s, a) = v_\eta(f_\xi(s)) + a_\psi(f_\xi(s), a) - \frac{\sum_{a'} a_\psi(f_\xi(s), a')}{N_{\text{actions}}},$$

其中 ξ 、 η 和 ψ 分别是共享编码器 f_ξ 、价值流 v_η 和优势流 a_ψ 的参数； $\theta = \{\xi, \eta, \psi\}$ 是它们的串联。

多步骤学习。Q-learning 累积单个奖励，然后在下一步使用贪婪动作进行引导。或者，可以使用前视 *multi-step* 目标（Sutton 1988）。我们将给定状态 S_t 的截断 n 步返回定义为

$$R_t^{(n)} \equiv \sum_{k=0}^{n-1} \gamma_t^{(k)} R_{t+k+1}. \quad (2)$$

然后通过最小化替代损失来定义 DQN 的多步变体，

$$(R_t^{(n)} + \gamma_t^{(n)} \max_{a'} q_\theta(S_{t+n}, a') - q_\theta(S_t, A_t))^2.$$

适当调整 n 的多步骤目标通常会带来更快的学习速度（Sutton 和 Barto 1998）。

分布式强化学习。我们可以学习近似回报分布而不是预期回报。最近，Bellemare、Dabney 和 Munos (2017) 提出用离散支撑 z 上的概率质量对此类分布进行建模，其中 z 是具有 $N_{\text{atoms}} \in \mathbb{N}^+$ *atoms* 的向量，由 $z^i = v_{\min} + (i-1) \frac{v_{\max} - v_{\min}}{N_{\text{atoms}} - 1}$ 为 $i \in \{1, \dots, N_{\text{atoms}}\}$ 定义。在此支持上定义时间 t 的近似分布 d_t ，每个原子 i 上的概率质量 $p_\theta^i(S_t, A_t)$ ，使得 $d_t = (z, p_\theta(S_t, A_t))$ 。目标是更新 θ ，使该分布与实际的回报分布紧密匹配。

要了解概率质量，关键的见解是回报分布满足贝尔曼方程的变体。对于给定的状态 S_t 和动作 A_t ，最优策略 π^* 下的回报分布应该与目标分布相匹配，该目标分布是通过获取下一个状态 S_{t+1} 和动作 $a_{t+1}^* = \pi^*(S_{t+1})$ 的分布来定义的，根据折扣将其收缩到零，并通过奖励（或随机情况下的奖励分配）将其移动。然后，通过首先构建目标分布的新支持，然后最小化分布 d_t 和目标分布 $d_t' \equiv (R_{t+1} + \gamma_{t+1} z, p_\theta(S_{t+1}, a_{t+1}^*))$ 之间的 Kullback-Leibler 散度，导出 Q 学习的分布变体，

$$D_{\text{KL}}(\Phi_z d_t' || d_t). \quad (3)$$

这里， Φ_z 是目标分布在固定支持 z 上的 L2 投影，而 $a_{t+1}^* = \arg\max_a q_\theta(S_{t+1}, a)$ 是相对于状态 S_{t+1} 中的平均动作值 $q_\theta(S_{t+1}, a) = z^\top p_\theta(S_{t+1}, a)$ 的贪婪动作。

与非分布情况一样，_我们可以使用参数 θ 的冻结副本来构造目标分布。参数化分布可以由神经网络表示，如 DQN 中，但具有 $N_{\text{atoms}} \times N_{\text{actions}}$ 输出。softmax 独立应用于输出的每个动作维度，以确保每个动作的分布得到适当的标准化。

喧闹的网。在《蒙特祖玛的复仇》等游戏中，使用 ϵ 贪婪策略进行探索的局限性很明显，在游戏中必须执行许多动作才能获得第一个奖励。噪声网络（Fortunato et al. 2017）提出了一个噪声线性层，它结合了确定性流和噪声流，

$$y = (b + Wx) + (b_{\text{noisy}} \odot \epsilon^b + (W_{\text{noisy}} \odot \epsilon^w)x), \quad (4)$$

其中 ϵ^b 和 ϵ^w 是随机变量， $\odot$ 表示逐元素乘积。然后可以使用此变换来代替标准线性 $y = b + Wx$ 。随着时间的推移，网络可以学会忽略噪声流，但会在状态空间的不同部分以不同的速率执行此操作，从而允许以自我退火的形式进行状态条件探索。

综合代理

在本文中，我们将所有上述组件集成到一个集成代理中，我们将其称为 *Rainbow*。

首先，我们用多步变体替换 1 步分布损失 (3)。我们根据累积折扣收缩 S_{t+n} 中的值分布，并将其移动截断的 n 步折扣回报，从而构建目标分布。这对应于将目标分布定义为 $d_t^{(n)} = (R_t^{(n)} + \gamma_t^{(n)} z, p_\theta(S_{t+n}, a_{t+n}^*))$ 。由此产生的损失是

$$D_{\text{KL}}(\Phi_z d_t^{(n)} || d_t),$$

其中， Φ_z 是到 z 的投影。

我们通过使用根据 *online network* 选择的 S_{t+n} 中的贪婪动作作为引导动作 a_{t+n}^* ，并使用 *target network* 评估该动作，将多步分布损失与双 Q 学习相结合。

在标准比例优先重放中（Schaul et al. 2015），绝对 TD 误差用于对转换进行优先级排序。这可以在分布设置中使用平均动作值来计算。然而，在我们的实验中，所有分布式 *Rainbow* 变体都通过 KL 损失来优先考虑转换，因为这是算法正在最小化的：

$$p_t \propto \left(D_{\text{KL}}(\Phi_z d_t^{(n)} || d_t) \right)^\omega.$$

作为优先级的 KL 损失对于嘈杂的随机环境可能更稳健，因为即使回报不确定，损失也会继续减少。

该网络架构是一种适合与返回分布一起使用的决斗网络架构。网络

has a shared representation $f_\xi(s)$, which is then fed into a value stream v_η with N_{atoms} outputs, and into an advantage stream a_ξ with $N_{\text{atoms}} \times N_{\text{actions}}$ outputs, where $a_\xi^i(f_\xi(s), a)$ will denote the output corresponding to atom i and action a . For each atom z^i , the value and advantage streams are aggregated, as in dueling DQN, and then passed through a softmax layer to obtain the normalised parametric distributions used to estimate the returns' distributions:

$$p_\theta^i(s, a) = \frac{\exp(v_\eta^i(\phi) + a_\psi^i(\phi, a) - \bar{a}_\psi^i(s))}{\sum_j \exp(v_\eta^j(\phi) + a_\psi^j(\phi, a) - \bar{a}_\psi^j(s))},$$

where $\phi = f_\xi(s)$ and $\bar{a}_\psi^i(s) = \frac{1}{N_{\text{actions}}} \sum_{a'} a_\psi^i(\phi, a')$.

We then replace all linear layers with their noisy equivalent described in Equation (4). Within these noisy linear layers we use factorised Gaussian noise (Fortunato et al. 2017) to reduce the number of independent noise variables.

Experimental Methods

We now describe the methods and setup used for configuring and evaluating the learning agents.

Evaluation Methodology. We evaluated all agents on 57 Atari 2600 games from the arcade learning environment (Bellemare et al. 2013). We follow the training and evaluation procedures of Mnih et al. (2015) and van Hasselt et al. (2016). The average scores of the agent are evaluated during training, every 1M steps in the environment, by suspending learning and evaluating the latest agent for 500K frames. Episodes are truncated at 108K frames (or 30 minutes of simulated play), as in van Hasselt et al. (2016).

Agents' scores are normalized, per game, so that 0% corresponds to a random agent and 100% to the average score of a human expert. Normalized scores can be aggregated across all Atari levels to compare the performance of different agents. It is common to track the *median* human normalized performance across all games. We also consider the number of games where the agent's performance is above some fraction of human performance, to disentangle where improvements in the median come from. The *mean* human normalized performance is potentially less informative, as it is dominated by a few games (e.g., Atlantis) where agents achieve scores orders of magnitude higher than humans do.

Besides tracking the median performance as a function of environment steps, at the end of training we re-evaluate the best agent snapshot using two different testing regimes. In the *no-ops starts* regime, we insert a random number (up to 30) of no-op actions at the beginning of each episode (as we do also in training). In the *human starts* regime, episodes are initialized with points randomly sampled from the initial portion of human expert trajectories (Nair et al. 2015); the difference between the two regimes indicates the extent to which the agent has over-fit to its own trajectories.

Due to space constraints, we focus on aggregate results across games. However, in the appendix we provide full learning curves for all games and all agents, as well as detailed comparison tables of raw and normalized scores, in both the no-op and human starts testing regimes.

Hyper-parameter tuning. All Rainbow's components have a number of hyper-parameters. The combinatorial space of hyper-parameters is too large for an exhaustive search, therefore we have performed limited tuning. For each component, we started with the values used in the paper that introduced this component, and tuned the most sensitive among hyper-parameters by manual coordinate descent.

DQN and its variants do not perform learning updates during the first 200K frames, to ensure sufficiently uncorrelated updates. We have found that, with prioritized replay, it is possible to start learning sooner, after only 80K frames.

DQN starts with an exploration ϵ of 1, corresponding to acting uniformly at random; it anneals the amount of exploration over the first 4M frames, to a final value of 0.1 (lowered to 0.01 in later variants). Whenever using Noisy Nets, we acted fully greedily ($\epsilon = 0$), with a value of 0.5 for the σ_0 hyper-parameter used to initialize the weights in the noisy stream¹. For agents without Noisy Nets, we used ϵ -greedy but decreased the exploration rate faster than was previously used, annealing ϵ to 0.01 in the first 250K frames.

We used the Adam optimizer (Kingma and Ba 2014), which we found less sensitive to the choice of the learning rate than RMSProp. DQN uses a learning rate of $\alpha = 0.00025$. In all Rainbow's variants we used a learning rate of $\alpha/4$, selected among $\{\alpha/2, \alpha/4, \alpha/6\}$, and a value of 1.5×10^{-4} for Adam's ϵ hyper-parameter.

For replay prioritization we used the recommended proportional variant, with priority exponent ω of 0.5, and linearly increased the importance sampling exponent β from 0.4 to 1 over the course of training. The priority exponent ω was tuned comparing values of $\{0.4, 0.5, 0.7\}$. Using the KL loss of distributional DQN as priority, we have observed that performance is very robust to the choice of ω .

The value of n in multi-step learning is a sensitive hyper-parameter of Rainbow. We compared values of $n = 1, 3$, and 5. We observed that both $n = 3$ and 5 did well initially, but overall $n = 3$ performed the best by the end.

The hyper-parameters (see Table 1) are identical across all 57 games, i.e., the Rainbow agent really is a *single* agent setup that performs well across all the games.

¹The noise was generated on the GPU. Tensorflow noise generation can be unreliable on GPU. If generating the noise on the CPU, lowering σ_0 to 0.1 may be helpful.

Parameter	Value
Min history to start learning	80K frames
Adam learning rate	0.0000625
Exploration ϵ	0.0
Noisy Nets σ_0	0.5
Target Network Period	32K frames
Adam ϵ	1.5×10^{-4}
Prioritization type	proportional
Prioritization exponent ω	0.5
Prioritization importance sampling β	0.4 $\rightarrow$ 1.0
Multi-step returns n	3
Distributional atoms	51
Distributional min/max values	$[-10, 10]$

Table 1: Rainbow hyper-parameters

有一个共享表示 $f_\xi(s)$ ，然后将其馈送到具有 N_{atoms} 输出的值流 v_η 中，以及具有 $N_{\text{atoms}} \times N_{\text{actions}}$ 输出的优势流 a_ξ 中，其中 $a_\xi^i(f_\xi(s), a)$ 将表示对应于原子 i 和操作 a 的输出。对于每个原子 z^i ，价值流和优势流被聚合，就像在决斗 DQN 中一样，然后通过 softmax 层以获得用于估计回报分布的归一化参数分布：

$$p_\theta^i(s, a) = \frac{\exp(v_\eta^i(\phi) + a_\psi^i(\phi, a) - \bar{a}_\psi^i(s))}{\sum_j \exp(v_\eta^j(\phi) + a_\psi^j(\phi, a) - \bar{a}_\psi^j(s))},$$

其中 $\phi = f_\xi(s)$ 和 $\bar{a}_\psi^i(s) = \frac{1}{N_{\text{actions}}} \sum_{a'} a_\psi^i(\phi, a')$ 。

然后，我们用等式 (4) 中描述的噪声等效层替换所有线性层。在这些噪声线性层中，我们使用因式分解高斯噪声 (Fortunato et al. 2017) 来减少独立噪声变量的数量。

实验方法

我们现在描述用于配置和评估学习代理的方法和设置。

评估方法。我们在街机学习环境中评估了 57 款 Atari 2600 游戏的所有智能体 (Bellemare 等人, 2013 年)。我们遵循 Mnih 等人的培训和评估程序。 (2015) 和范哈塞尔特等人。 (2016)。在训练期间，环境中每 1M 步，通过暂停学习和评估 500K 帧的最新代理来评估代理的平均分数。剧集以 108K 帧 (或 30 分钟的模拟播放) 被截断，如 van Hasselt 等人所述。 (2016)。

每场比赛的智能体得分都被标准化，因此 0% 对应于随机智能体，100% 对应于人类专家的平均得分。可以汇总所有 Atari 级别的归一化分数，以比较不同代理的性能。跟踪所有游戏中的 *median* 人类标准化表现是很常见的。我们还考虑了智能体表现高于人类表现一部分的游戏数量，以理清中位数改进的来源。 *mean* 人类标准化表现可能信息量较少，因为它主要由一些游戏 (例如，亚特兰蒂斯) 主导，其中智能体的得分比人类高出几个数量级。

除了跟踪作为环境步骤函数的中值性能之外，在训练结束时，我们使用两种不同的测试方案重新评估最佳代理快照。在 *no-ops starts* 机制中，我们在每集开始时插入随机数量 (最多 30 个) 的无操作动作 (正如我们在训练中所做的那样)。在 *human starts* 体系中，情节是用从人类专家轨迹的初始部分随机采样的点来初始化的 (Nair et al. 2015)；两种制度之间的差异表明了智能体对其自身轨迹的过度拟合程度。

由于空间限制，我们专注于跨游戏的汇总结果。然而，在附录中，我们提供了所有游戏和所有代理的完整学习曲线，以及在无操作和人类启动测试制度下原始分数和标准化分数的详细比较表。

超参数调整。Rainbow 的所有组件都有许多超参数。超参数的组合空间对于穷举搜索来说太大，因此我们进行了有限的调整。对于每个组件，我们从介绍该组件的论文中使用的值开始，并通过手动坐标下降调整超参数中最敏感的值。

DQN 及其变体在前 200K 帧期间不执行学习更新，以确保足够的不相关更新。我们发现，通过优先重播，可以在仅 80K 帧后更快地开始学习。

DQN 从探索 ϵ 为 1 开始，对应于随机均匀行动；它将前 4M 帧的探索量退火到最终值 0.1 (在以后的变体中降低到 0.01)。每当使用噪声网络时，我们都表现得完全贪婪 ($\epsilon = 0$)，用于初始化噪声流¹中权重的 σ_0 超参数值为 0.5。对于没有噪声网络的智能体，我们使用 ϵ -greedy，但降低探索速率的速度比之前使用的更快，在前 250K 帧中将 ϵ 退火到 0.01。

我们使用了 Adam 优化器 (Kingma 和 Ba 2014)，我们发现它对学习率的选择不如 RMSProp 敏感。DQN 使用的学习率为 $\alpha = 0.00025$ 在 Rainbow 的所有变体中，我们使用的学习率为 $\alpha/4$ ，从 $\{\alpha/2, \alpha/4, \alpha/6\}$ 中选择，Adam 的 ϵ 超参数值为 1.5×10^{-4} 。

对于重放优先级，我们使用推荐的比例变量，优先级指数 ω 为 0.5，并在训练过程中将重要性采样指数 β 从 0.4 线性增加到 1。优先级指数 ω 通过比较 $\{0.4, 0.5, 0.7\}$ 的值进行调整。使用分布式 DQN 的 KL 损失作为优先级，我们观察到性能对于 ω 的选择非常稳健。

多步学习中 n 的值是 Rainbow 的敏感超参数。我们比较了 $n = 1, 3$ 和 5 的值。我们观察到 $n = 3$ 和 5 最初都表现良好，但总体而言 $n = 3$ 最终表现最好。

所有 57 款游戏的超参数 (参见表 1) 都是相同的，即 Rainbow 代理确实是一个在所有游戏中都表现良好的 *single* 代理设置。

¹The noise was generated on the GPU. Tensorflow noise generation can be unreliable on GPU. If generating the noise on the CPU, lowering σ_0 to 0.1 may be helpful.

Parameter	Value
Min history to start learning	80K frames
Adam learning rate	0.0000625
Exploration ϵ	0.0
Noisy Nets σ_0	0.5
Target Network Period	32K frames
Adam ϵ	1.5×10^{-4}
Prioritization type	proportional
Prioritization exponent ω	0.5
Prioritization importance sampling β	0.4 $\rightarrow$ 1.0
Multi-step returns n	3
Distributional atoms	51
Distributional min/max values	$[-10, 10]$

表1: Rainbow超参数

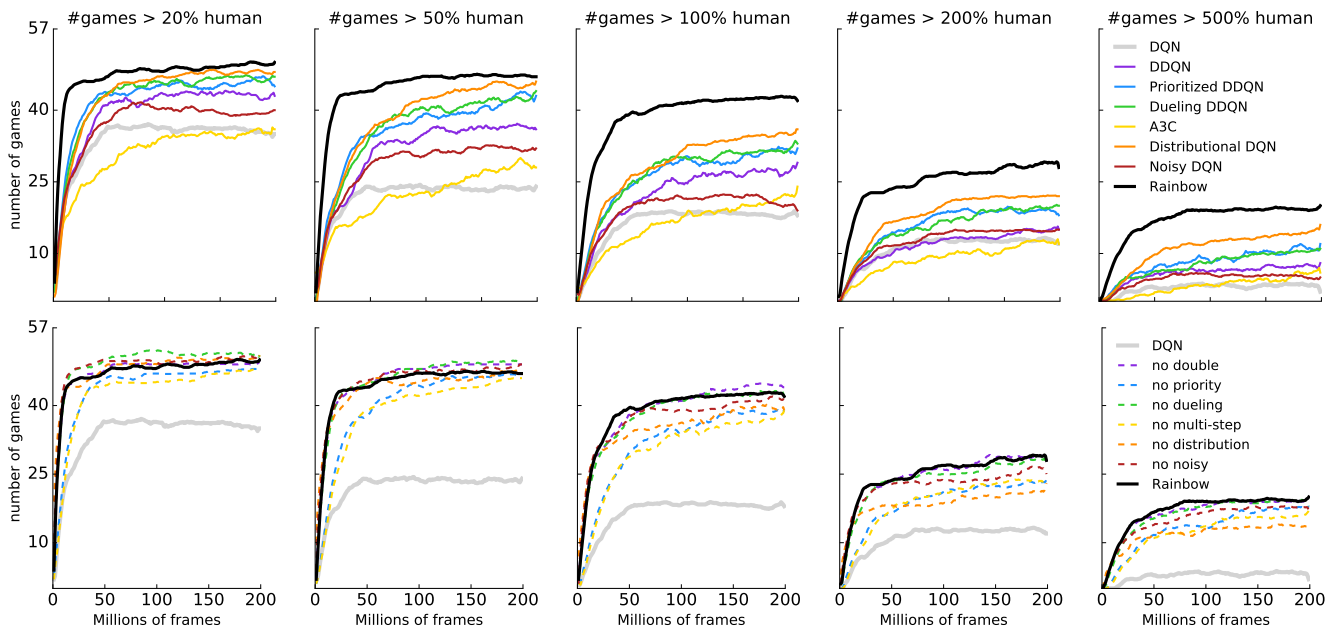


Figure 2: Each plot shows, for several agents, the number of games where they have achieved at least a given fraction of human performance, as a function of time. From left to right we consider the 20%, 50%, 100%, 200% and 500% thresholds. On the first row we compare Rainbow to the baselines. On the second row we compare Rainbow to its ablations.

Analysis

In this section we analyse the main experimental results. First, we show that Rainbow compares favorably to several published agents. Then we perform ablation studies, comparing several variants of the agent, each corresponding to removing a single component from Rainbow.

Comparison to published baselines. In Figure 1 we compare the Rainbow’s performance (measured in terms of the median human normalized score across games) to the corresponding curves for A3C, DQN, DDQN, Prioritized DDQN, Dueling DDQN, Distributional DQN, and Noisy DQN. We thank the authors of the Dueling and Prioritized agents for providing the learning curves of these, and report our own re-runs for DQN, A3C, DDQN, Distributional DQN and Noisy DQN. The performance of Rainbow is significantly better than any of the baselines, both in data efficiency, as well as in final performance. Note that we match final performance of DQN after 7M frames, surpass the best final performance of these baselines in 44M frames, and reach substantially improved final performance.

In the final evaluations of the agent, after the end of training, Rainbow achieves a median score of 223% in the no-ops regime; in the human starts regime we measured a median score of 153%. In Table 2 we compare these scores to the published median scores of the individual baselines.

In Figure 2 (top row) we plot the number of games where an agent has reached some specified level of human normalized performance. From left to right, the subplots show on how many games the different agents have achieved 20%, 50%, 100%, 200% and 500% human normalized perfor-

mance. This allows us to identify where the overall improvements in performance come from. Note that the gap in performance between Rainbow and other agents is apparent at all levels of performance: the Rainbow agent is improving scores on games where the baseline agents were already good, as well as improving in games where baseline agents are still far from human performance.

Learning speed. As in the original DQN setup, we ran each agent on a single GPU. The 7M frames required to match DQN’s final performance correspond to less than 10 hours of wall-clock time. A full run of 200M frames corresponds to approximately 10 days, and this varies by less than 20% between all of the discussed variants. The litera-

Agent	no-ops	human starts
DQN	79%	68%
DDQN (*)	117%	110%
Prioritized DDQN (*)	140%	128%
Dueling DDQN (*)	151%	117%
A3C (*)	-	116%
Noisy DQN	118%	102%
Distributional DQN	164%	125%
Rainbow	223%	153%

Table 2: Median normalized scores of the best agent snapshots for Rainbow and baselines. For methods marked with an asterisk, the scores come from the corresponding publication. DQN’s scores comes from the dueling networks paper, since DQN’s paper did not report scores for all 57 games. The others scores come from our own implementations.

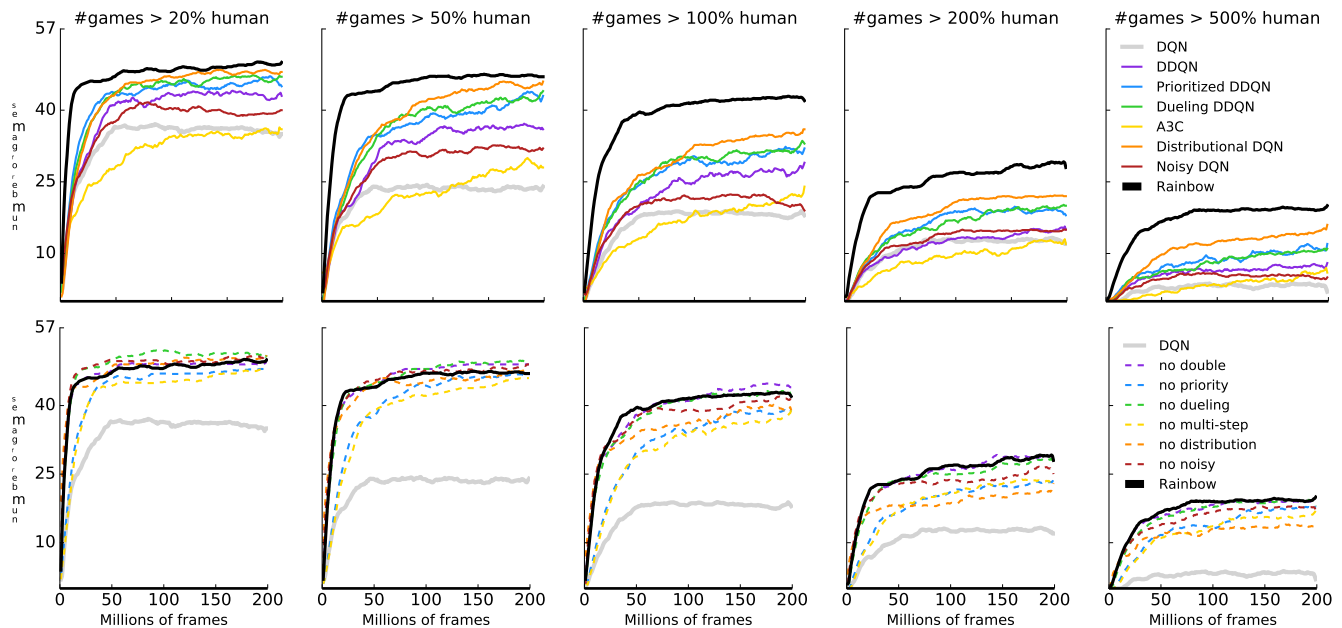


图 2: 每个图显示了多个智能体至少达到人类表现的给定分数的游戏数量 (作为时间的函数)。从左到右, 我们考虑 20%、50%、100%、200% 和 500% 阈值。在第一行, 我们将 Rainbow 与基线进行比较。在第二行, 我们将 Rainbow 与其消融进行比较。

分析

本节我们分析主要实验结果。首先, 我们表明 Rainbow 与几个已发布的代理相比具有优势。然后我们进行消融研究, 比较该试剂的几种变体, 每种变体对应于从 Rainbow 中去除一个组件。

与已发布的基线进行比较。在图 1 中, 我们将 Rainbow 的性能 (以跨游戏的人类归一化得分中位数来衡量) 与 A3C、DQN、DDQN、优先 DDQN、决斗 DDQN、分布式 DQN 和噪声 DQN 的相应曲线进行了比较。我们感谢 Dueling 和 Prioritized 代理的作者提供了这些代理的学习曲线, 并报告了我们自己对 DQN、A3C、DDQN、分布式 DQN 和噪声 DQN 的重新运行。Rainbow 的性能明显优于任何基线, 无论是在数据效率还是最终性能方面。请注意, 我们在 7M 帧后匹配 DQN 的最终性能, 超越这些基线在 44M 帧中的最佳最终性能, 并达到显著提高的最终性能。

在训练结束后对智能体的最终评估中, Rainbow 在无操作状态下的中位得分为 223%; 在人类起始体系中, 我们测得的中位分数为 153%。在表 2 中, 我们将这些分数与已发布的各个基线的中值分数进行了比较。

在图 2 (顶行) 中, 我们绘制了智能体达到人类标准化表现的特定水平的游戏数量。从左到右, 子图显示了不同智能体在多少场比赛中达到了人类标准化表现的 20%、50%、100%、200% 和 500%。

曼斯。这使我们能够确定性能的整体改进来自何处。请注意, Rainbow 和其他智能体之间的性能差距在各个性能水平上都很明显: Rainbow 智能体正在提高基准智能体已经很好的游戏的分数, 以及在基准智能体与人类表现仍相距甚远的游戏中的得分。

学习速度。与原始 DQN 设置一样, 我们在单个 GPU 上运行每个代理。匹配 DQN 最终性能所需的 7M 帧相当于不到 10 小时的挂钟时间。完整运行 200M 帧大约需要 10 天, 并且所有讨论的变体之间的差异不到 20%。文学——

Agent	no-ops	human starts
DQN	79%	68%
DDQN (*)	117%	110%
Prioritized DDQN (*)	140%	128%
Dueling DDQN (*)	151%	117%
A3C (*)	-	116%
Noisy DQN	118%	102%
Distributional DQN	164%	125%
Rainbow	223%	153%

表 2: Rainbow 和基线的最佳代理快照的中值归一化分数。对于标有星号的方法, 分数来自相应的出版物。DQN 的分数来自 Dueling Networks 论文, 因为 DQN 的论文没有报告所有 57 场比赛的分数。其他分数来自我们自己的实现。

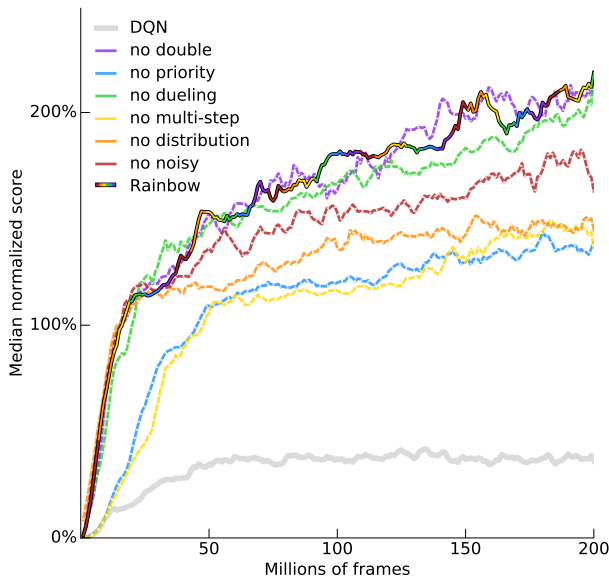


Figure 3: **Median human-normalized performance** across 57 Atari games, as a function of time. We compare our integrated agent (rainbow-colored) to DQN (gray) and to six different ablations (dashed lines). Curves are smoothed with a moving average over 5 points.

ture contains many alternative training setups that improve performance as a function of wall-clock time by exploiting parallelism, e.g., Nair et al. (2015), Salimans et al. (2017), and Mnih et al. (2016). Properly relating the performance across such very different hardware/compute resources is non-trivial, so we focused exclusively on algorithmic variations, allowing apples-to-apples comparisons. While we consider them to be important and complementary, we leave questions of scalability and parallelism to future work.

Ablation studies. Since Rainbow integrates several different ideas into a single agent, we conducted additional experiments to understand the contribution of the various components, in the context of this specific combination.

To gain a better understanding of the contribution of each component to the Rainbow agent, we performed ablation studies. In each ablation, we removed one component from the full Rainbow combination. Figure 3 shows a comparison for median normalized score of the full Rainbow to six ablated variants. Figure 2 (bottom row) shows a more detailed breakdown of how these ablations perform relative to different thresholds of human normalized performance, and Figure 4 shows the gain or loss from each ablation for every game, averaged over the full learning run.

Prioritized replay and multi-step learning were the two most crucial components of Rainbow, in that removing either component caused a large drop in median performance. Unsurprisingly, the removal of either of these hurt early performance. Perhaps more surprisingly, the removal of multi-step learning also hurt final performance. Zooming in on individual games (Figure 4), we see both components helped

almost uniformly across games (the full Rainbow performed better than either ablation in 53 games out of 57).

Distributional Q-learning ranked immediately below the previous techniques for relevance to the agent’s performance. Notably, in early learning no difference is apparent, as shown in Figure 3, where for the first 40 million frames the distributional-ablation performed as well as the full agent. However, without distributions, the performance of the agent then started lagging behind. When the results are separated relatively to human performance in Figure 2, we see that the distributional-ablation primarily seems to lag on games that are above human level or near it.

In terms of median performance, the agent performed better when Noisy Nets were included; when these are removed and exploration is delegated to the traditional ϵ -greedy mechanism, performance was worse in aggregate (red line in Figure 3). While the removal of Noisy Nets produced a large drop in performance for several games, it also provided small increases in other games (Figure 4).

In aggregate, we did not observe a significant difference when removing the dueling network from the full Rainbow. The median score, however, hides the fact that the impact of Dueling differed between games, as shown by Figure 4. Figure 2 shows that Dueling perhaps provided some improvement on games with above-human performance levels (# games > 200%), and some degradation on games with sub-human performance (# games > 20%).

Also in the case of double Q-learning, the observed difference in median performance (Figure 3) is limited, with the component sometimes harming or helping depending on the game (Figure 4). To further investigate the role of double Q-learning, we compared the predictions of our trained agents to the actual discounted returns computed from clipped rewards. Comparing Rainbow to the agent where double Q-learning was ablated, we observed that the actual returns are often higher than 10 and therefore fall outside the support of the distribution, spanning from -10 to $+10$. This leads to underestimated returns, rather than overestimations. We hypothesize that clipping the values to this constrained range counteracts the overestimation bias of Q-learning. Note, however, that the importance of double Q-learning may increase if the support of the distributions is expanded.

In the appendix, for each game we show final performance and learning curves for Rainbow, its ablations, and baselines.

Discussion

We have demonstrated that several improvements to DQN can be successfully integrated into a single learning algorithm that achieves state-of-the-art performance. Moreover, we have shown that within the integrated algorithm, all but one of the components provided clear performance benefits. There are many more algorithmic components that we were not able to include, which would be promising candidates for further experiments on integrated agents. Among the many possible candidates, we discuss several below.

We have focused here on value-based methods in the Q-learning family. We have not considered purely policy-based RL algorithms such as trust-region policy optimisa-

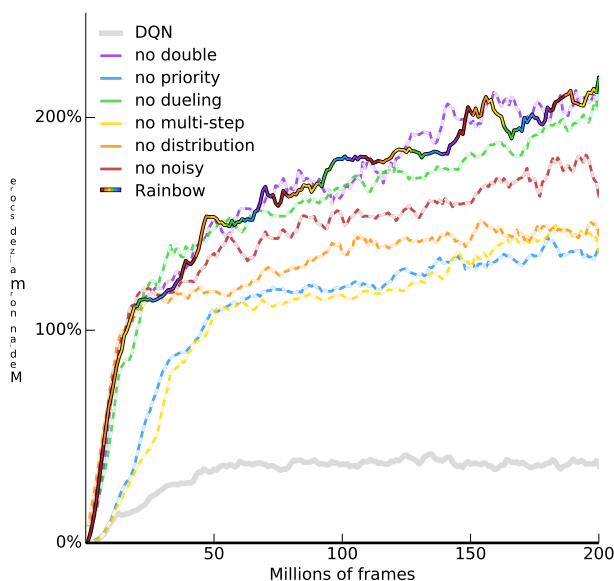


图 3: 57 款 Atari 游戏的人类标准化表现中位数, 作为时间的函数。我们将我们的集成代理 (彩虹色) 与 DQN (灰色) 和六种不同的消融 (虚线) 进行比较。曲线使用超过 5 个点的移动平均值进行平滑。

ture 包含许多替代训练设置, 通过利用并行性来提高作为挂钟时间函数的性能, 例如 Nair 等人。(2015), 萨利曼斯等人。(2017) 和 Mnih 等人。(2016)。在如此不同的硬件/计算资源之间正确关联性能并非易事, 因此我们专门关注算法变化, 以便进行同类比较。虽然我们认为它们是重要且互补的, 但我们将可扩展性和并行性问题留给了未来的工作。

消融研究。由于 Rainbow 将几种不同的想法集成到一个代理中, 因此我们进行了额外的实验, 以了解不同组件在这种特定组合的背景下的贡献。

为了更好地了解每种成分对 Rainbow 剂的贡献, 我们进行了消融研究。在每次消融中, 我们从完整的 Rainbow 组合中删除了一个组件。图 3 显示了完整 Rainbow 与六个消融变体的中值归一化分数的比较。图 2 (底行) 更详细地显示了这些消融相对于人类标准化表现的不同阈值的表现, 图 4 显示了每场游戏的每次消融的增益或损失, 在整个学习过程中取平均值。

优先重放和多步学习是 Rainbow 的两个最关键的组成部分, 因为删除任何一个组件都会导致中值性能大幅下降。毫不奇怪, 删除其中任何一个都会损害早期性能。也许更令人惊讶的是, 取消多步骤学习也会损害最终表现。放大单个游戏 (图 4), 我们发现这两个组件都有帮助

各场比赛几乎一致 (在 57 场比赛中, 完整的 Rainbow 比任何一种消融的表现都要好)。

就与智能体性能的相关性而言, 分布式 Q 学习的排名直接低于之前的技术。值得注意的是, 在早期学习中没有明显的差异, 如图 3 所示, 在前 4000 万帧中, 分布式消融的表现与完整智能体一样好。然而, 如果没有分布, 代理的性能就会开始落后。当结果相对于图 2 中的人类表现分开时, 我们发现分布消融似乎主要滞后于高于或接近人类水平的游戏。

就中值性能而言, 当包含 Noisy Nets 时, 智能体表现更好; 当这些被删除并将探索委托给传统的 ϵ - 贪婪机制时, 总体性能会更差 (图 3 中的红线)。虽然移除 Noisy Nets 导致一些游戏的性能大幅下降, 但它也为其他游戏带来了小幅提升 (图 4)。

总的来说, 当从完整的 Rainbow 中删除决斗网络时, 我们没有观察到显着的差异。然而, 中位数分数掩盖了决斗对游戏之间的影响不同的事实, 如图 4 所示。图 2 显示, 决斗可能对高于人类表现水平的游戏提供了一些改进 ($\# \text{ games} > 200\%$), 但对低于人类表现水平的游戏带来了一些退化 ($\# \text{ games} > 20\%$)。

同样在双 Q 学习的情况下, 观察到的中值性能差异 (图 3) 是有限的, 该组件有时会损害或帮助, 具体取决于游戏 (图 4)。为了进一步研究双 Q 学习的作用, 我们将训练有素的代理的预测与根据削减的奖励计算出的实际折扣回报进行了比较。将 Rainbow 与取消双 Q 学习的智能体进行比较, 我们观察到实际回报通常高于 10, 因此超出了分布的支持范围, 范围从 -10 到 $+10$ 。这会导致回报被低估, 而不是高估。我们假设将值限制在这个受限范围内可以抵消 Q 学习的高估偏差。但请注意, 如果分布的支持得到扩展, 双 Q 学习的重要性可能会增加。

在附录中, 我们展示了每个游戏的 Rainbow 的最终性能和学习曲线、其消融和基线。

讨论

我们已经证明, 对 DQN 的多项改进可以成功集成到单个学习算法中, 从而实现最先进的性能。此外, 我们已经证明, 在集成算法中, 除了一个组件之外, 所有组件都提供了明显的性能优势。还有更多我们无法包含的算法组件, 这些组件可能是集成代理进一步实验的有希望的候选者。在众多可能的候选者中, 我们在下面讨论其中的几个。

我们在这里重点关注 Q-learning 系列中基于价值的方法。我们还没有考虑纯粹基于策略的强化学习算法, 例如信任区域策略优化

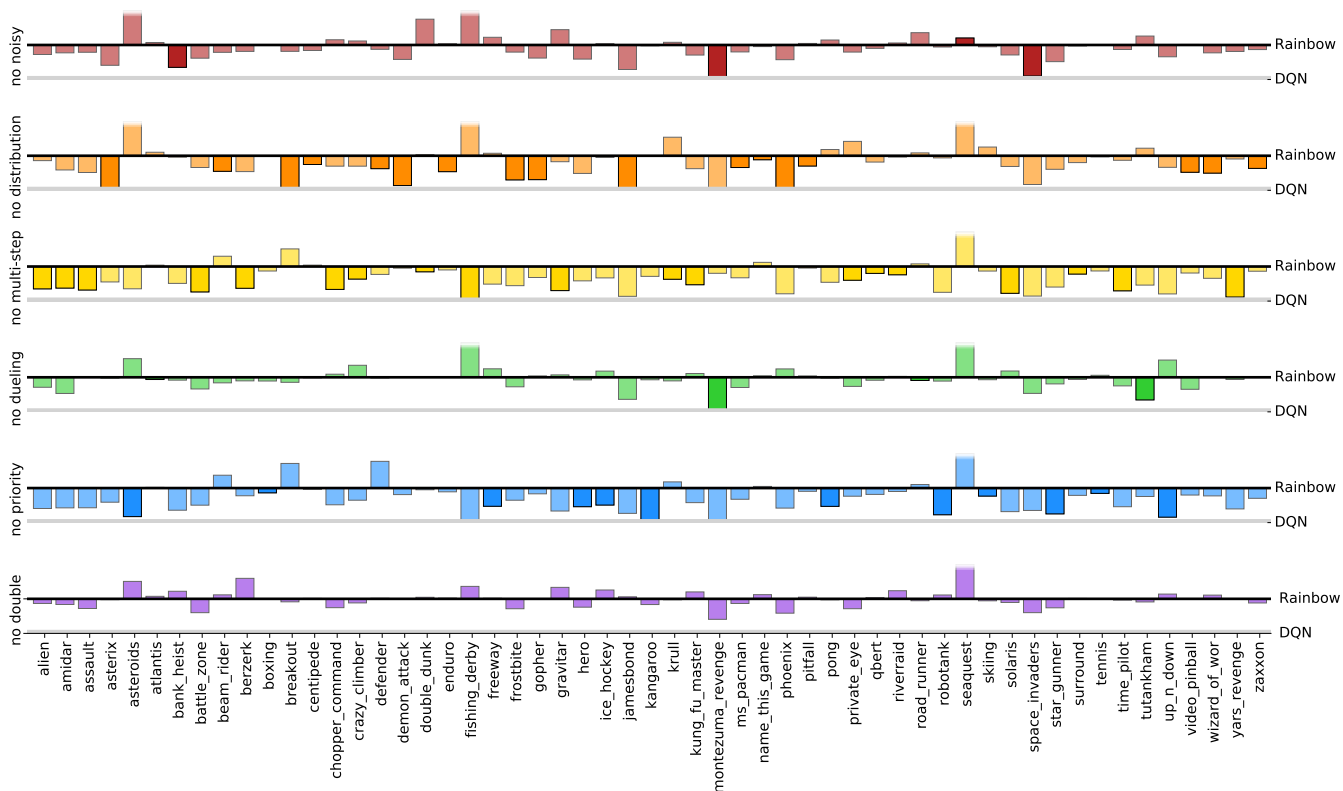


Figure 4: **Performance drops of ablation agents** on all 57 Atari games. Performance is the area under the learning curve, normalized relative to the Rainbow agent and DQN. Two games where DQN outperforms Rainbow are omitted. The ablation leading to the strongest drop is highlighted for each game. The removal of either prioritization or multi-step learning reduces performance across most games, but the contribution of each component varies substantially per game.

tion (Schulman et al. 2015), nor actor-critic methods (Mnih et al. 2016; O’Donoghue et al. 2016).

A number of algorithms exploit a sequence of data to achieve improved learning efficiency. Optimality tightening (He et al. 2016) uses multi-step returns to construct additional inequality bounds, instead of using them to replace the 1-step targets used in Q-learning. Eligibility traces allow a soft combination over n -step returns (Sutton 1988). However, sequential methods all leverage more computation per gradient than the multi-step targets used in Rainbow. Furthermore, introducing prioritized sequence replay raises questions of how to store, replay and prioritise sequences.

Episodic control (Blundell et al. 2016) also focuses on data efficiency, and was shown to be very effective in some domains. It improves early learning by using episodic memory as a complementary learning system, capable of immediately re-enacting successful action sequences.

Besides Noisy Nets, numerous other exploration methods could also be useful algorithmic ingredients: among these Bootstrapped DQN (Osband et al. 2016), intrinsic motivation (Stadie, Levine, and Abbeel 2015) and count-based exploration (Bellemare et al. 2016). Integration of these alternative components is fruitful subject for further research.

In this paper we have focused on the core learning up-

dates, without exploring alternative computational architectures. Asynchronous learning from parallel copies of the environment, as in A3C (Mnih et al. 2016), Gorila (Nair et al. 2015), or Evolution Strategies (Salimans et al. 2017), can be effective in speeding up learning, at least in terms of wall-clock time. Note, however, they can be less data efficient.

Hierarchical RL has also been applied with success to several complex Atari games. Among successful applications of HRL we highlight h-DQN (Kulkarni et al. 2016a) and Feudal Networks (Vezhnevets et al. 2017).

The state representation could also be made more efficient by exploiting auxiliary tasks such as pixel control or feature control (Jaderberg et al. 2016), supervised predictions (Dosovitskiy and Koltun 2016) or successor features (Kulkarni et al. 2016b).

To evaluate Rainbow fairly against the baselines, we have followed the common domain modifications of clipping rewards, fixed action-repetition, and frame-stacking, but these might be removed by other learning algorithm improvements. Pop-Art normalization (van Hasselt et al. 2016) allows reward clipping to be removed, while preserving a similar level of performance. Fine-grained action repetition (Sharma, Lakshminarayanan, and Ravindran 2017) enabled to learn how to repeat actions. A recurrent state network

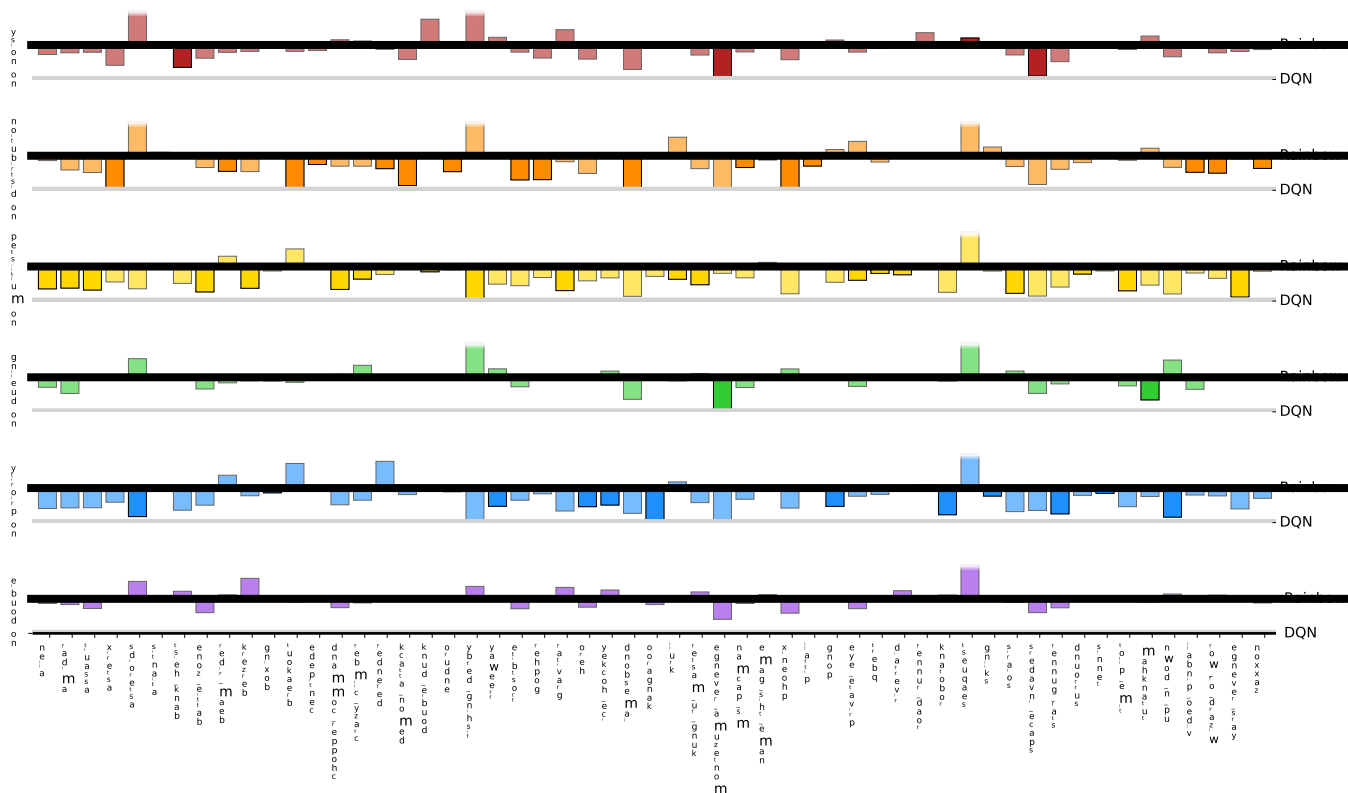


图 4: 所有 57 款 Atari 游戏中消融剂的性能下降。性能是学习曲线下的面积，相对于 Rainbow 代理和 DQN 进行标准化。省略了 DQN 优于 Rainbow 的两场比赛。每场比赛都会突出显示导致最强下降的消融。取消优先级或多步骤学习会降低大多数游戏的性能，但每个组件的贡献在每个游戏中都有很大差异。

化 (Schulman et al. 2015)，也不是演员批评家方法 (Mnih et al. 2016; O’Donoghue et al. 2016)。

许多算法利用一系列数据来提高学习效率。最优性紧缩 (He et al. 2016) 使用多步返回来构建额外的不等式边界，而不是用它们来代替 Q 学习中使用的 1 步目标。资格轨迹允许对 n 步回报进行软组合 (Sutton 1988)。然而，与 Rainbow 中使用的多步目标相比，顺序方法在每个梯度上都利用了更多的计算量。此外，引入优先序列重放提出了如何存储、重放序列和确定序列优先级的问题。

情景控制 (Blundell et al. 2016) 也注重数据效率，并且在某些领域被证明非常有效。它通过使用情景记忆作为补充学习系统来改善早期学习，能够立即重新制定成功的动作序列。

除了噪声网络之外，许多其他探索方法也可能是有用的算法成分：其中包括 Bootstrapped DQN (Osband 等人, 2016 年)、内在动机 (Stadie, Levine 和 Abbeel, 2015 年) 和基于计数的探索 (Bellemare 等人, 2016 年)。这些替代组件的集成是进一步研究的富有成效的课题。

在本文中，我们重点关注核心学习内容——

日期，而不探索替代的计算架构。从环境的并行副本进行异步学习，如 A3C (Mnih 等人, 2016 年)、Gorila (Nair 等人, 2015 年) 或进化策略 (Salimans 等人, 2017 年)，可以有效地加快学习速度，至少在挂钟时间方面是这样。但请注意，它们的数据效率可能较低。

分层强化学习也已成功应用于多种复杂的 Atari 游戏。在 HRL 的成功应用中，我们重点关注 h-DQN (Kulkarni 等人, 2016a) 和 Feudal Networks (Vezhnevets 等人, 2017)。

通过利用辅助任务，例如像素控制或特征控制 (Jaderberg et al. 2016)、监督预测 (Dosovitskiy and Koltun 2016) 或后继特征 (Kulkarni et al. 2016b)，状态表示也可以变得更加高效。

为了根据基线公平地评估 Rainbow，我们遵循了剪辑奖励、固定动作重复和帧堆叠等常见领域的修改，但这些可能会被其他学习算法的改进所删除。Pop-Art 标准化 (van Hasselt et al. 2016) 允许删除奖励剪辑，同时保持类似的性能水平。细粒度的动作重复 (Sharma, Lakshminarayanan 和 Ravindran 2017) 能够学习如何重复动作。循环状态网络

(Hausknecht and Stone 2015) can learn a temporal state representation, replacing the fixed stack of observation frames. In general, we believe that exposing the real game to the agent is a promising direction for future research.

References

- Bellemare, M. G.; Naddaf, Y.; Veness, J.; and Bowling, M. 2013. The arcade learning environment: An evaluation platform for general agents. *J. Artif. Intell. Res. (JAIR)* 47:253–279.
- Bellemare, M. G.; Srinivasan, S.; Ostrovski, G.; Schaul, T.; Saxton, D.; and Munos, R. 2016. Unifying count-based exploration and intrinsic motivation. In *NIPS*.
- Bellemare, M. G.; Dabney, W.; and Munos, R. 2017. A distributional perspective on reinforcement learning. In *ICML*.
- Blundell, C.; Uria, B.; Pritzel, A.; Li, Y.; Ruderman, A.; Leibo, J. Z.; Rae, J.; Wierstra, D.; and Hassabis, D. 2016. Model-Free Episodic Control. *ArXiv e-prints*.
- Dosovitskiy, A., and Koltun, V. 2016. Learning to act by predicting the future. *CoRR* abs/1611.01779.
- Fortunato, M.; Azar, M. G.; Piot, B.; Menick, J.; Osband, I.; Graves, A.; Mnih, V.; Munos, R.; Hassabis, D.; Pietquin, O.; Blundell, C.; and Legg, S. 2017. Noisy networks for exploration. *CoRR* abs/1706.10295.
- Hausknecht, M., and Stone, P. 2015. Deep recurrent Q-learning for partially observable MDPs. *arXiv preprint arXiv:1507.06527*.
- He, F. S.; Liu, Y.; Schwing, A. G.; and Peng, J. 2016. Learning to play in a day: Faster deep reinforcement learning by optimality tightening. *CoRR* abs/1611.01606.
- Jaderberg, M.; Mnih, V.; Czarnecki, W. M.; Schaul, T.; Leibo, J. Z.; Silver, D.; and Kavukcuoglu, K. 2016. Reinforcement learning with unsupervised auxiliary tasks. *CoRR* abs/1611.05397.
- Kingma, D. P., and Ba, J. 2014. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.
- Kulkarni, T. D.; Narasimhan, K.; Saeedi, A.; and Tenenbaum, J. B. 2016a. Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. *CoRR* abs/1604.06057.
- Kulkarni, T. D.; Saeedi, A.; Gautam, S.; and Gershman, S. J. 2016b. Deep successor reinforcement learning. *arXiv preprint arXiv:1606.02396*.
- Lin, L.-J. 1992. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning* 8(3):293–321.
- Mnih, V.; Kavukcuoglu, K.; Silver, D.; Graves, A.; Antonoglou, I.; Wierstra, D.; and Riedmiller, M. A. 2013. Playing atari with deep reinforcement learning. *CoRR* abs/1312.5602.
- Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; Petersen, S.; Beattie, C.; Sadik, A.; Antonoglou, I.; King, H.; Kumaran, D.; Wierstra, D.; Legg, S.; and Hassabis, D. 2015. Human-level control through deep reinforcement learning. *Nature* 518(7540):529–533.
- Mnih, V.; Badia, A. P.; Mirza, M.; Graves, A.; Lillicrap, T.; Harley, T.; Silver, D.; and Kavukcuoglu, K. 2016. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning*.
- Nair, A.; Srinivasan, P.; Blackwell, S.; Alcicek, C.; Fearon, R.; De Maria, A.; Panneershelvam, V.; Suleyman, M.; Beattie, C.; Petersen, S.; Legg, S.; Mnih, V.; Kavukcuoglu, K.; and Silver, D. 2015. Massively parallel methods for deep reinforcement learning. *arXiv preprint arXiv:1507.04296*.
- O’Donoghue, B.; Munos, R.; Kavukcuoglu, K.; and Mnih, V. 2016. Pq: Combining policy gradient and q-learning. *CoRR* abs/1611.01626.
- Osband, I.; Blundell, C.; Pritzel, A.; and Roy, B. V. 2016. Deep exploration via bootstrapped dqn. In *NIPS*.
- Salimans, T.; Ho, J.; Chen, X.; and Sutskever, I. 2017. Evolution strategies as a scalable alternative to reinforcement learning. *CoRR* abs/1703.03864.
- Schaul, T.; Quan, J.; Antonoglou, I.; and Silver, D. 2015. Prioritized experience replay. In *Proc. of ICLR*.
- Schulman, J.; Levine, S.; Moritz, P.; Jordan, M.; and Abbeel, P. 2015. Trust region policy optimization. In *Proceedings of the 32nd International Conference on International Conference on Machine Learning - Volume 37, ICML’15*, 1889–1897. JMLR.org.
- Sharma, S.; Lakshminarayanan, A. S.; and Ravindran, B. 2017. Learning to repeat: Fine grained action repetition for deep reinforcement learning. *arXiv preprint arXiv:1702.06054*.
- Stadie, B. C.; Levine, S.; and Abbeel, P. 2015. Incentivizing exploration in reinforcement learning with deep predictive models. *CoRR* abs/1507.00814.
- Sutton, R. S., and Barto, A. G. 1998. *Reinforcement Learning: An Introduction*. The MIT press, Cambridge MA.
- Sutton, R. S. 1988. Learning to predict by the methods of temporal differences. *Machine learning* 3(1):9–44.
- Tieleman, T., and Hinton, G. 2012. Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude. *COURSERA: Neural networks for machine learning* 4(2):26–31.
- van Hasselt, H.; Guez, A.; Guez, A.; Hessel, M.; Mnih, V.; and Silver, D. 2016. Learning values across many orders of magnitude. In *Advances in Neural Information Processing Systems* 29, 4287–4295.
- van Hasselt, H.; Guez, A.; and Silver, D. 2016. Deep reinforcement learning with double Q-learning. In *Proc. of AAAI*, 2094–2100.
- van Hasselt, H. 2010. Double Q-learning. In *Advances in Neural Information Processing Systems* 23, 2613–2621.
- Vezhnevets, A. S.; Osindero, S.; Schaul, T.; Heess, N.; Jaderberg, M.; Silver, D.; and Kavukcuoglu, K. 2017. Feudal networks for hierarchical reinforcement learning. *CoRR* abs/1703.01161.

(Hausknecht 和 Stone 2015) 可以学习时间状态表示, 取代固定的观察帧堆栈。总的来说, 我们认为将真实的游戏暴露给代理是未来研究的一个有前途的方向。

参考

贝尔马尔, M.G.; 纳达夫, Y.; 维内斯, J.; 和 Bowling, M. 2013. 街机学习环境: 一般代理的评估平台。 *J. Artif. Intell. Res. (JAIR)* 47:253–279. 贝尔马尔, M.G.; 斯里尼瓦桑, S.; 奥斯特洛夫斯基, G.; 斯考尔, T.; 萨克斯顿, D.; Munos, R. 2016. 统一基于计数的探索和内在动机。在 *NIPS* 中。贝尔马尔, M.G.; 达布尼, W.; Munos, R. 2017. 强化学习的分布视角。在 *ICML* 中。布伦德尔, C.; 乌里亚, B.; 普里策尔, A.; 李, Y. 鲁德曼, A.; 雷波, J.Z.; 雷伊, J.; 维尔斯特拉, D.; 和 Hassabis, D. 2016. 无模型情景控制。 *ArXiv e-prints*. Dosovitskiy, A. 和 Koltun, V. 2016. 通过预测未来来学习行动。 *CoRR* 绝对/1611.01779. 福尔图纳托, M.; 阿扎尔, M.G.; 皮奥特, B.; 梅尼克, J.; 奥斯本, I.; 格雷夫斯, A.; 姆尼赫, V.; 穆诺斯, R.; 哈萨比斯, D.; 皮特金, O.; 布伦德尔, C.; 和 Legg, S. 2017. 用于探索的嘈杂网络。 *CoRR* 绝对/1706.10295. Hausknecht, M. 和 Stone, P. 2015. 部分可观察 MDP 的深度循环 Q 学习。 *arXiv preprint arXiv:1507.06527*. 他, F.S.; 刘, Y. 施温格, A.G.; 和 Peng, J. 2016. 一天学会玩: 通过最优性紧缩加快深度强化学习。 *CoRR* 绝对/1611.01606. 贾德伯格, M.; 姆尼赫, V.; Czarnecki, W.M.; 斯考尔, T.; 雷波, J.Z.; 银, D.; 和 Kavukcuoglu, K. 2016. 无监督辅助任务的强化学习。 *CoRR* 绝对/1611.05397. Kingma, D. P. 和 Ba, J. 2014. Adam: 一种随机优化方法。在 *Proceedings of the 3rd International Conference on Learning Representations (ICLR)* 中。库尔卡尼, T.D.; 纳拉西姆汗, K.; 萨伊迪, A.; 和 Tenenbaum, J. B. 2016a. 分层深度强化学习: 整合时间抽象和内在动机。 *CoRR* 绝对/1604.06057. 库尔卡尼, T.D.; 萨伊迪, A.; 高塔姆, S.; 和 Gershman, S. J. 2016b. 深度后继强化学习。 *arXiv preprint arXiv:1606.02396*. 林, L.-J. 1992. 基于强化学习、规划和教学的自我改进反应代理。 *Machine Learning* 8(3):293–321. 姆尼赫, V.; 卡武克库奥卢, K.; 银, D.; 格雷夫斯, A.; 安东诺格鲁, I.; 维尔斯特拉, D.; Riedmiller, M. A. 2013. 通过深度强化学习玩 atari。 *CoRR* 绝对/1312.5602. 姆尼赫, V.; 卡武克库奥卢, K.; 银, D.; 鲁苏, A.A.; 维内斯, J.; 贝尔马尔, M.G.; 格雷夫斯, A.; 里德米勒, M.; 菲杰兰, A.K.; 奥斯特洛夫斯基, G.; 彼得森, S.; 比蒂, C.; 萨迪克, A.; 安东诺格鲁, I.; 金, H.; 库马兰, D.; 维尔斯特拉, D.; 莱格,

S.; 和 Hassabis, D. 2015. 通过深度强化学习实现人类水平的控制。 *Nature* 518 (7540): 529–533. 姆尼赫, V.; 巴迪亚, A.P.; 米尔扎, M.; 格雷夫斯, A.; 利利克拉普, T.; 哈雷, T.; 银, D.; 和 Kavukcuoglu, K. 2016. 深度强化学习的异步方法。在 *International Conference on Machine Learning* 中。奈尔, A.; 斯里尼瓦桑, P.; 布莱克威尔, S.; 阿尔切克, C.; 费伦, R.; 德玛丽亚, A.; 潘内斯谢尔瓦姆, V.; 苏莱曼, M.; Beat-tie, C.; 彼得森, S.; 莱格, S.; 姆尼赫, V.; 卡武克库奥卢, K.; Silver, D. 2015. 深度强化学习的大规模并行方法。 *arXiv preprint arXiv:1507.04296*. 奥多诺霍, B.; 穆诺斯, R.; 卡武克库奥卢, K.; 和 Mnih, V. 2016. Pqg: 结合策略梯度和 q 学习。 *CoRR* 绝对/1611.01626. 奥斯本, I.; 布伦德尔, C.; 普里策尔, A.; 和 Roy, B. V. 2016. 通过 bootstrapped dqn 进行深度探索。在 *NIPS* 中。萨利曼斯, T.; 何, J. 陈 X.; Sutskever, I. 2017. 进化策略作为强化学习的可扩展替代方案。 *CoRR* 绝对/1703.03864. 斯考尔, T.; 全, J.; 安东诺格鲁, I.; Silver, D. 2015. 优先体验重放。在 *Proc. of ICLR* 中。舒尔曼, J.; 莱文, S.; 莫里茨, P.; 乔丹, M.; 和 Abbeel, P. 2015. 信任区域政策优化。在 *Proceedings of the 32nd International Conference on International Conference on Machine Learning - Volume 37, ICML' 15*, 1889–1897. JMLR.org. 夏尔马, S.; 拉克什米纳拉亚南 (Lakshminarayanan), A.S.; Ravindran, B. 2017. 学习重复: 深度强化学习的细粒度动作重复。 *arXiv preprint arXiv:1702.06054*. 斯塔迪, 不列颠哥伦比亚省; 莱文, S.; 和 Abbeel, P. 2015. 利用深度预测模型激励强化学习的探索。 *CoRR* 绝对/1507.00814. Sutton, R. S. 和 Barto, A. G. 1998. *Reinforcement Learning: An Introduction*. 麻省理工学院出版社, 马萨诸塞州剑桥。Sutton, R. S. 1988. 学习通过时间差异方法进行预测。 *Machine learning* 3(1):9–44. Tieleman, T. 和 Hinton, G. 2012. 讲座 6.5-rmsprop: 将梯度除以其最近幅度的运行平均值。 *COURSERA: Neural networks for machine learning* 4(2):26–31. 范哈瑟尔特, H.; 格斯, A.; 格斯, A.; 赫塞尔, M.; 姆尼赫, V.; Silver, D. 2016. 跨多个数量级的学习价值。在 *Advances in Neural Information Processing Systems* 29, 4287–4295. 范哈瑟尔特, H.; 格斯, A.; Silver, D. 2016. 采用双 Q 学习的深度强化学习。在 *Proc. of AAAI*, 2094–2100. van Hasselt, H. 2010. 双 Q 学习。在 *Advances in Neural Information Processing Systems* 23, 2613–2621. Vezhnevets, A.S.; 奥辛德罗, S.; 斯考尔, T.; 赫斯, N.; Jaderberg, M.; 银, D.; 和 Kavukcuoglu, K. 2017. 用于分层强化学习的封建网络。 *CoRR* 绝对/1703.01161.

Wang, Z.; Schaul, T.; Hessel, M.; van Hasselt, H.; Lanctot, M.; and de Freitas, N. 2016. Dueling network architectures for deep reinforcement learning. In *Proceedings of The 33rd International Conference on Machine Learning*, 1995–2003.

王, Z. 斯考尔, T.; 赫塞尔, M.; 范哈瑟尔特, H.; 兰克托, M.; 和 de Freitas, N. 2016。深度强化学习的决斗网络架构。在*Proceedings of The 33rd International Conference on Machine Learning*, 1995-2003。

Appendix

Table 3 lists the preprocessing of environment frames, rewards and discounts introduced by DQN. Table 4 lists the additional hyper-parameters that Rainbow inherits from DQN and the other baselines considered in this paper. The hyper-parameters for which Rainbow uses non standard settings are instead listed in the main text. In the subsequent pages, we list the tables showing, for each game, the score achieved by Rainbow and several baselines in both the no-ops regime (Table 6) and the human-starts regime (Table 5). In Figures 5 and 6 we also plot, for each game, the learning curves of Rainbow, several baselines, and all ablation experiments. These learning curves are smoothed with a moving average over a window of 10.

Hyper-parameter	value
Grey-scaling	True
Observation down-sampling	(84, 84)
Frames stacked	4
Action repetitions	4
Reward clipping	[-1, 1]
Terminal on loss of life	True
Max frames per episode	108K

Table 3: **Preprocessing:** the values of these hyper-parameters are the same used by DQN and its variants. They are here listed for completeness. Observations are grey-scaled and rescaled to 84×84 pixels. 4 consecutive frames are concatenated as each state’s representation. Each action selected by the agent is repeated for 4 times. Rewards are clipped between $-1, +1$. In games where the player has multiple lives, transitions associated to the loss of a life are considered terminal. All episodes are capped after 108K frames.

Hyper-parameter	value
Q network: channels	32, 64, 64
Q network: filter size	$8 \times 8, 4 \times 4, 3 \times 3$
Q network: stride	4, 2, 1
Q network: hidden units	512
Q network: output units	Number of actions
Discount factor	0.99
Memory size	1M transitions
Replay period	every 4 agent steps
Minibatch size	32

Table 4: **Additional hyper-parameters:** the values of these hyper-parameters are the same used by DQN and its variants. The network has 3 convolutional layers: with 32, 64 and 64 channels. The layers use $8 \times 8, 4 \times 4, 3 \times 3$ filters with strides of 4, 2, 1, respectively. The value and advantage streams of the dueling architecture have both a hidden layer with 512 units. The output layer of the network has a number of units equal to the number of actions available in the game. We use a discount factor of 0.99, which is set to 0 on terminal transitions. We perform a learning update every 4 agent steps, using mini-batches of 32 transitions.

附录

表3列出了DQN引入的环境框架、奖励和折扣的预处理。表4列出了 Rainbow 从 DQN 继承的附加超参数以及本文考虑的其他基线。Rainbow 使用非标准设置的超参数在正文中列出。在接下来的几页中，我们列出了表格，显示了每场比赛 Rainbow 取得的分数以及无操作制度（表6）和人工启动制度（表5）中的几个基线。在图5和图6中，我们还为每个游戏绘制了 Rainbow 的学习曲线、几个基线和所有消融实验。这些学习曲线通过10窗口内的移动平均值进行平滑。

Hyper-parameter	value
Grey-scaling	True
Observation down-sampling	(84, 84)
Frames stacked	4
Action repetitions	4
Reward clipping	[-1, 1]
Terminal on loss of life	True
Max frames per episode	108K

表3：预处理：这些超参数的值与 DQN 及其变体使用的值相同。此处列出它们是为了完整性。观察结果被灰度化并重新缩放为 84×84 像素。4 个连续帧连接起来作为每个状态的表示。代理选择的每个动作都会重复 4 次。奖励被限制在 $-1, +1$ 之间。在玩家拥有多条生命的游戏中，与生命损失相关的转换被视为终结。所有剧集均在 108K 帧之后达到上限。

Hyper-parameter	value
Q network: channels	32, 64, 64
Q network: filter size	$8 \times 8, 4 \times 4, 3 \times 3$
Q network: stride	4, 2, 1
Q network: hidden units	512
Q network: output units	Number of actions
Discount factor	0.99
Memory size	1M transitions
Replay period	every 4 agent steps
Minibatch size	32

表4：其他超参数：这些超参数的值与 DQN 及其变体使用的值相同。该网络有 3 个卷积层：32、64 和 64 个通道。这些层使用 8×8 、 4×4 、 3×3 滤波器，步长分别为 4、2、1。决斗架构的价值流和优势流均具有 512 个单元的隐藏层。网络输出层的单元数量等于游戏中可用动作的数量。我们使用折扣因子 0.99，在终端转换时设置为 0。我们使用 32 个转换的小批量，每 4 个代理步骤执行一次学习更新。

Game	DQN	A3C	DDQN	Prior. DDQN	Duel. DDQN	Distrib. DQN	Noisy DQN	Rainbow
alien	634.0	518.4	1033.4	900.5	1,486.5	1,997.5	533.3	6,022.9
amidar	178.4	263.9	169.1	218.4	172.7	237.7	148.0	202.8
assault	3489.3	5474.9	6060.8	7,748.5	3,994.8	5,101.3	5,124.3	14,491.7
asterix	3170.5	22140.5	16837.0	31,907.5	15,840.0	395,599.5	8,277.3	280,114.0
asteroids	1458.7	4474.5	1193.2	1,654.0	2,035.4	2,071.7	4,078.1	2,249.4
atlantis	292491.0	911,091.0	319688.0	593,642.0	445,360.0	289,803.0	303,666.5	814,684.0
bank_heist	312.7	970.1	886.0	816.8	1,129.3	835.6	955.0	826.0
battle_zone	23750.0	12950.0	24740.0	29,100.0	31,320.0	32,250.0	26,985.0	52,040.0
beam_rider	9743.2	22707.9	17417.2	26,172.7	14,591.3	15,002.4	15,241.5	21,768.5
berzerk	493.4	817.9	1011.1	1,165.6	910.6	1,000.0	670.8	1,793.4
bowling	56.5	35.1	69.6	65.8	65.7	76.8	79.3	39.4
boxing	70.3	59.8	73.5	68.6	77.3	62.1	66.3	54.9
breakout	354.5	681.9	368.9	371.6	411.6	548.7	423.3	379.5
centipede	3973.9	3755.8	3853.5	3,421.9	4,881.0	7,476.9	4,214.4	7,160.9
chopper_command	5017.0	7021.0	3495.0	6,604.0	3,784.0	9,600.5	8,778.5	10,916.0
crazy_climber	98128.0	112646.0	113782.0	131,086.0	124,566.0	154,416.5	98,576.5	143,962.0
defender	15917.5	56533.0	27510.0	21,093.5	33,996.0	32,246.0	18,037.5	47,671.3
demon_attack	12550.7	113,308.4	69803.4	73,185.8	56,322.8	109,856.6	25,207.8	109,670.7
double_dunk	-6.0	-0.1	-0.3	2.7	-0.8	-3.7	-1.0	-0.6
enduro	626.7	-82.5	1216.6	1,884.4	2,077.4	2,133.4	1,021.5	2,061.1
fishing_derby	-1.6	18.8	3.2	9.2	-4.1	-4.9	-3.7	22.6
freeway	26.9	0.1	28.8	27.9	0.2	28.8	27.1	29.1
frostbite	496.1	190.5	1448.1	2,930.2	2,332.4	2,813.9	418.8	4,141.1
gopher	8190.4	10022.8	15253.0	57,783.8	20,051.4	27,778.3	13,131.0	72,595.7
gravitar	298.0	303.5	200.5	218.0	297.0	422.0	250.5	567.5
hero	14992.9	32464.1	14892.5	20,506.4	15,207.9	28,554.2	2,454.2	50,496.8
ice_hockey	-1.6	-2.8	-2.5	-1.0	-1.3	-0.1	-2.4	-0.7
kanaroo	4496.0	94.0	11204.0	10,241.0	10,334.0	9,555.5	7,465.0	10,841.0
krull	6206.0	5560.0	6796.1	7,406.5	8,051.6	6,757.8	6,833.5	6,715.5
kung_fu_master	20882.0	28819.0	30207.0	31,244.0	24,288.0	33,890.0	27,921.0	28,999.8
montezuma_revenge	47.0	67.0	42.0	13.0	22.0	130.0	55.0	154.0
ms_pacman	1092.3	653.7	1241.3	1,824.6	2,250.6	2,064.1	1,012.1	2,570.2
name_this_game	6738.8	10476.1	8960.3	11,836.1	11,185.1	11,382.3	7,186.4	11,686.5
phoenix	7484.8	52894.1	12366.5	27,430.1	20,410.5	31,358.3	15,505.0	103,061.6
pitfall	-113.2	-78.5	-186.7	-14.8	-46.9	-342.8	-154.4	-37.6
pong	18.0	5.6	19.1	18.9	18.8	18.9	18.0	19.0
private_eye	207.9	206.9	-575.5	179.0	292.6	5,717.5	5,955.4	1,704.4
qbert	9271.5	15148.8	11020.8	11,277.0	14,175.8	15,035.9	9,176.6	18,397.6
road_runner	35215.0	34216.0	43156.0	56,990.0	58,549.0	56,086.0	35,376.5	54,261.0
robotank	58.7	32.8	59.1	55.4	62.0	49.8	50.9	55.2
seaquest	4216.7	2355.4	14498.0	39,096.7	37,361.6	3,275.4	2,353.1	19,176.0
skiing	-12142.1	-10911.1	-11490.4	-10,852.8	-11,928.0	-13,247.7	-13,905.9	-11,685.8
solaris	1295.4	1956.0	810.0	2,238.2	1,768.4	2,530.2	2,608.2	2,860.7
space_invaders	1293.8	15,730.5	2628.7	9,063.0	5,993.1	6,368.6	1,697.2	12,629.0
star_gunner	52970.0	138218.0	58365.0	51,959.0	90,804.0	67,054.5	31,864.5	123,853.0
surround	-6.0	-9.7	1.9	-0.9	4.0	4.5	-3.1	7.0
tennis	11.1	-6.3	-7.8	-2.0	4.4	22.6	-2.1	-2.2
time_pilot	4786.0	12,679.0	6608.0	7,448.0	6,601.0	7,684.5	5,311.0	11,190.5
tutankham	45.6	156.3	92.2	33.6	48.0	124.3	123.3	126.9
venture	136.0	23.0	21.0	244.0	200.0	462.0	10.5	45.0
video_pinball	154414.1	331628.1	367823.7	374,886.9	110,976.2	455,052.7	241,851.7	506,817.2
wizard_of_wor	1609.0	17,244.0	6201.0	7,451.0	7,054.0	11,824.5	4,796.5	14,631.5
yars_revenge	4577.5	7157.5	6270.6	5,965.1	25,976.5	8,267.7	5,487.3	93,007.9
zaxxon	4412.0	24,622.0	8593.0	9,501.0	10,164.0	15,130.0	7,650.5	19,658.0

Table 5: **Human Starts** evaluation regime: Raw scores across all games, averaged over 200 testing episodes, from the agent snapshot that obtained the highest score during training. We report the published scores for DQN, A3C, DDQN, Dueling DDQN, and Prioritized DDQN. For Distributional DQN and Rainbow we report our own evaluations of the agents.

Game	DQN	A3C	DDQN	Prior. DDQN	Duel. DDQN	Distrib. DQN	Noisy DQN	Rainbow
alien	634.0	518.4	1033.4	900.5	1,486.5	1,997.5	533.3	6,022.9
amidar	178.4	263.9	169.1	218.4	172.7	237.7	148.0	202.8
assault	3489.3	5474.9	6060.8	7,748.5	3,994.8	5,101.3	5,124.3	14,491.7
asterix	3170.5	22140.5	16837.0	31,907.5	15,840.0	395,599.5	8,277.3	280,114.0
asteroids	1458.7	4474.5	1193.2	1,654.0	2,035.4	2,071.7	4,078.1	2,249.4
atlantis	292491.0	911,091.0	319688.0	593,642.0	445,360.0	289,803.0	303,666.5	814,684.0
bank_heist	312.7	970.1	886.0	816.8	1,129.3	835.6	955.0	826.0
battle_zone	23750.0	12950.0	24740.0	29,100.0	31,320.0	32,250.0	26,985.0	52,040.0
beam_rider	9743.2	22707.9	17417.2	26,172.7	14,591.3	15,002.4	15,241.5	21,768.5
berzerk	493.4	817.9	1011.1	1,165.6	910.6	1,000.0	670.8	1,793.4
bowling	56.5	35.1	69.6	65.8	65.7	76.8	79.3	39.4
boxing	70.3	59.8	73.5	68.6	77.3	62.1	66.3	54.9
breakout	354.5	681.9	368.9	371.6	411.6	548.7	423.3	379.5
centipede	3973.9	3755.8	3853.5	3,421.9	4,881.0	7,476.9	4,214.4	7,160.9
chopper_command	5017.0	7021.0	3495.0	6,604.0	3,784.0	9,600.5	8,778.5	10,916.0
crazy_climber	98128.0	112646.0	113782.0	131,086.0	124,566.0	154,416.5	98,576.5	143,962.0
defender	15917.5	56533.0	27510.0	21,093.5	33,996.0	32,246.0	18,037.5	47,671.3
demon_attack	12550.7	113,308.4	69803.4	73,185.8	56,322.8	109,856.6	25,207.8	109,670.7
double_dunk	-6.0	-0.1	-0.3	2.7	-0.8	-3.7	-1.0	-0.6
enduro	626.7	-82.5	1216.6	1,884.4	2,077.4	2,133.4	1,021.5	2,061.1
fishing_derby	-1.6	18.8	3.2	9.2	-4.1	-4.9	-3.7	22.6
freeway	26.9	0.1	28.8	27.9	0.2	28.8	27.1	29.1
frostbite	496.1	190.5	1448.1	2,930.2	2,332.4	2,813.9	418.8	4,141.1
gopher	8190.4	10022.8	15253.0	57,783.8	20,051.4	27,778.3	13,131.0	72,595.7
gravitar	298.0	303.5	200.5	218.0	297.0	422.0	250.5	567.5
hero	14992.9	32464.1	14892.5	20,506.4	15,207.9	28,554.2	2,454.2	50,496.8
ice_hockey	-1.6	-2.8	-2.5	-1.0	-1.3	-0.1	-2.4	-0.7
kangaroo	4496.0	94.0	11204.0	10,241.0	10,334.0	9,555.5	7,465.0	10,841.0
krull	6206.0	5560.0	6796.1	7,406.5	8,051.6	6,757.8	6,833.5	6,715.5
kung_fu_master	20882.0	28819.0	30207.0	31,244.0	24,288.0	33,890.0	27,921.0	28,999.8
montezuma_revenge	47.0	67.0	42.0	13.0	22.0	130.0	55.0	154.0
ms_pacman	1092.3	653.7	1241.3	1,824.6	2,250.6	2,064.1	1,012.1	2,570.2
name_this_game	6738.8	10476.1	8960.3	11,836.1	11,185.1	11,382.3	7,186.4	11,686.5
phoenix	7484.8	52894.1	12366.5	27,430.1	20,410.5	31,358.3	15,505.0	103,061.6
pitfall	-113.2	-78.5	-186.7	-14.8	-46.9	-342.8	-154.4	-37.6
pong	18.0	5.6	19.1	18.9	18.8	18.9	18.0	19.0
private_eye	207.9	206.9	-575.5	179.0	292.6	5,717.5	5,955.4	1,704.4
qbert	9271.5	15148.8	11020.8	11,277.0	14,175.8	15,035.9	9,176.6	18,397.6
road_runner	35215.0	34216.0	43156.0	56,990.0	58,549.0	56,086.0	35,376.5	54,261.0
robotank	58.7	32.8	59.1	55.4	62.0	49.8	50.9	55.2
sequest	4216.7	2355.4	14498.0	39,096.7	37,361.6	3,275.4	2,353.1	19,176.0
skiing	-12142.1	-10911.1	-11490.4	-10,852.8	-11,928.0	-13,247.7	-13,905.9	-11,685.8
solaris	1295.4	1956.0	810.0	2,238.2	1,768.4	2,530.2	2,608.2	2,860.7
space_invaders	1293.8	15,730.5	2628.7	9,063.0	5,993.1	6,368.6	1,697.2	12,629.0
star_gunner	52970.0	138218.0	58365.0	51,959.0	90,804.0	67,054.5	31,864.5	123,853.0
surround	-6.0	-9.7	1.9	-0.9	4.0	4.5	-3.1	7.0
tennis	11.1	-6.3	-7.8	-2.0	4.4	22.6	-2.1	-2.2
time_pilot	4786.0	12,679.0	6608.0	7,448.0	6,601.0	7,684.5	5,311.0	11,190.5
tutankham	45.6	156.3	92.2	33.6	48.0	124.3	123.3	126.9
venture	136.0	23.0	21.0	244.0	200.0	462.0	10.5	45.0
video_pinball	154414.1	331628.1	367823.7	374,886.9	110,976.2	455,052.7	241,851.7	506,817.2
wizard_of_wor	1609.0	17,244.0	6201.0	7,451.0	7,054.0	11,824.5	4,796.5	14,631.5
yars_revenge	4577.5	7157.5	6270.6	5,965.1	25,976.5	8,267.7	5,487.3	93,007.9
zaxxon	4412.0	24,622.0	8593.0	9,501.0	10,164.0	15,130.0	7,650.5	19,658.0

表 5: Human Starts 评估制度: 所有游戏的原始分数, 平均超过 200 个测试集, 来自训练期间获得最高分数的智能体快照。我们报告 DQN、A3C、DDQN、Dueling DDQN 和 Prioritized DDQN 的已发布分数。对于分布式 DQN 和 Rainbow, 我们报告了我们自己对代理的评估。

Game	DQN	DDQN	Prior. DDQN	Duel. DDQN	Distrib. DQN	Noisy DQN	Rainbow
alien	1620.0	3747.7	6,648.6	4,461.4	4,055.8	2,394.9	9,491.7
amidar	978.0	1793.3	2,051.8	2,354.5	1,267.9	1,608.0	5,131.2
assault	4280.0	5393.2	7,965.7	4,621.0	5,909.0	5,198.6	14,198.5
asterix	4359.0	17356.5	41,268.0	28,188.0	400,529.5	12,403.8	428,200.3
asteroids	1364.5	734.7	1,699.3	2,837.7	2,354.7	4,814.1	2,712.8
atlantis	279987.0	106056.0	427,658.0	382,572.0	273,895.0	329,010.0	826,659.5
bank_heist	455.0	1030.6	1,126.8	1,611.9	1,056.7	1,323.0	1,358.0
battle_zone	29900.0	31700.0	38,130.0	37,150.0	41,145.0	32,050.0	62,010.0
beam_rider	8627.5	13772.8	22,430.7	12,164.0	13,213.4	12,534.0	16,850.2
berzerk	585.6	1225.4	1,614.2	1,472.6	1,421.8	837.3	2,545.6
bowling	50.4	68.1	62.6	65.5	74.1	77.3	30.0
boxing	88.0	91.6	98.8	99.4	98.1	83.3	99.6
breakout	385.5	418.5	381.5	345.3	612.5	459.1	417.5
centipede	4657.7	5409.4	5,175.4	7,561.4	9,015.5	4,355.8	8,167.3
chopper_command	6126.0	5809.0	5,135.0	11,215.0	13,136.0	9,519.0	16,654.0
crazy_climber	110763.0	117282.0	183,137.0	143,570.0	178,355.0	118,768.0	168,788.5
defender	23633.0	35338.5	24,162.5	42,214.0	37,896.8	23,083.0	55,105.0
demon_attack	12149.4	58044.2	70,171.8	60,813.3	110,626.5	24,950.1	111,185.2
double_dunk	-6.6	-5.5	4.8	0.1	-3.8	-1.8	-0.3
enduro	729.0	1211.8	2,155.0	2,258.2	2,259.3	1,129.2	2,125.9
fishing_derby	-4.9	15.5	30.2	46.4	9.1	7.7	31.3
freeway	30.8	33.3	32.9	0.0	33.6	32.0	34.0
frostbite	797.4	1683.3	3,421.6	4,672.8	3,938.2	583.6	9,590.5
gopher	8777.4	14840.8	49,097.4	15,718.4	28,841.0	15,107.9	70,354.6
gravitar	473.0	412.0	330.5	588.0	681.0	443.5	1,419.3
hero	20437.8	20130.2	27,153.9	20,818.2	33,860.9	5,053.1	55,887.4
ice_hockey	-1.9	-2.7	0.3	0.5	1.3	-2.1	1.1
kangaroo	7259.0	12992.0	14,492.0	14,854.0	12,909.0	12,117.0	14,637.5
krull	8422.3	7920.5	10,263.1	11,451.9	9,885.9	9,061.9	8,741.5
kung_fu_master	26059.0	29710.0	43,470.0	34,294.0	43,009.0	34,099.0	52,181.0
montezuma_revenge	0.0	0.0	0.0	0.0	367.0	0.0	384.0
ms_pacman	3085.6	2711.4	4,751.2	6,283.5	3,769.2	2,501.6	5,380.4
name_this_game	8207.8	10616.0	13,439.4	11,971.1	12,983.6	8,332.4	13,136.0
phoenix	8485.2	12252.5	32,808.3	23,092.2	34,775.0	16,974.3	108,528.6
pitfall	-286.1	-29.9	0.0	0.0	-2.1	-18.2	0.0
pong	19.5	20.9	20.7	21.0	20.8	21.0	20.9
private_eye	146.7	129.7	200.0	103.0	15,172.9	3,966.0	4,234.0
qbert	13117.3	15088.5	18,802.8	19,220.3	16,956.0	15,276.3	33,817.5
road_runner	39544.0	44127.0	62,785.0	69,524.0	63,366.0	41,681.0	62,041.0
robotank	63.9	65.1	58.6	65.3	54.2	53.5	61.4
sequest	5860.6	16452.7	44,417.4	50,254.2	4,754.4	2,495.4	15,898.9
skiing	-13062.3	-9021.8	-9,900.5	-8,857.4	-14,959.8	-16,307.3	-12,957.8
solaris	3482.8	3067.8	1,710.8	2,250.8	5,643.1	3,204.5	3,560.3
space_invaders	1692.3	2525.5	7,696.9	6,427.3	6,869.1	2,145.5	18,789.0
star_gunner	54282.0	60142.0	56,641.0	89,238.0	69,306.5	34,504.5	127,029.0
surround	-5.6	-2.9	2.1	4.4	6.2	-3.3	9.7
tennis	12.2	-22.8	0.0	5.1	23.6	0.0	-0.0
time_pilot	4870.0	8339.0	11,448.0	11,666.0	7,875.0	6,157.0	12,926.0
tutankham	68.1	218.4	87.2	211.4	249.4	231.6	241.0
venture	163.0	98.0	863.0	497.0	1,107.0	0.0	5.5
video_pinball	196760.4	309941.9	406,420.4	98,209.5	478,646.7	270,444.6	533,936.5
wizard_of_wor	2704.0	7492.0	10,373.0	7,855.0	15,994.5	5,432.0	17,862.5
yars_revenge	18089.9	11712.6	16,451.7	49,622.1	16,608.6	9,570.1	102,557.0
zaxxon	5363.0	10163.0	13,490.0	12,944.0	18,347.5	9,390.0	22,209.5

Table 6: **No-op starts** evaluation regime: Raw scores across all games, averaged over 200 testing episodes, from the agent snapshot that obtained the highest score during training. We report the published scores for DQN, DDQN, Dueling DDQN, and Prioritized DDQN. For Distributional DQN and Rainbow we report our own evaluations of the agents. A3C is not listed since the paper did not report the scores for the no-ops regime.

Game	DQN	DDQN	Prior. DDQN	Duel. DDQN	Distrib. DQN	Noisy DQN	Rainbow
alien	1620.0	3747.7	6,648.6	4,461.4	4,055.8	2,394.9	9,491.7
amidar	978.0	1793.3	2,051.8	2,354.5	1,267.9	1,608.0	5,131.2
assault	4280.0	5393.2	7,965.7	4,621.0	5,909.0	5,198.6	14,198.5
asterix	4359.0	17356.5	41,268.0	28,188.0	400,529.5	12,403.8	428,200.3
asteroids	1364.5	734.7	1,699.3	2,837.7	2,354.7	4,814.1	2,712.8
atlantis	279987.0	106056.0	427,658.0	382,572.0	273,895.0	329,010.0	826,659.5
bank_heist	455.0	1030.6	1,126.8	1,611.9	1,056.7	1,323.0	1,358.0
battle_zone	29900.0	31700.0	38,130.0	37,150.0	41,145.0	32,050.0	62,010.0
beam_rider	8627.5	13772.8	22,430.7	12,164.0	13,213.4	12,534.0	16,850.2
berzerk	585.6	1225.4	1,614.2	1,472.6	1,421.8	837.3	2,545.6
bowling	50.4	68.1	62.6	65.5	74.1	77.3	30.0
boxing	88.0	91.6	98.8	99.4	98.1	83.3	99.6
breakout	385.5	418.5	381.5	345.3	612.5	459.1	417.5
centipede	4657.7	5409.4	5,175.4	7,561.4	9,015.5	4,355.8	8,167.3
chopper_command	6126.0	5809.0	5,135.0	11,215.0	13,136.0	9,519.0	16,654.0
crazy_climber	110763.0	117282.0	183,137.0	143,570.0	178,355.0	118,768.0	168,788.5
defender	23633.0	35338.5	24,162.5	42,214.0	37,896.8	23,083.0	55,105.0
demon_attack	12149.4	58044.2	70,171.8	60,813.3	110,626.5	24,950.1	111,185.2
double_dunk	-6.6	-5.5	4.8	0.1	-3.8	-1.8	-0.3
enduro	729.0	1211.8	2,155.0	2,258.2	2,259.3	1,129.2	2,125.9
fishing_derby	-4.9	15.5	30.2	46.4	9.1	7.7	31.3
freeway	30.8	33.3	32.9	0.0	33.6	32.0	34.0
frostbite	797.4	1683.3	3,421.6	4,672.8	3,938.2	583.6	9,590.5
gopher	8777.4	14840.8	49,097.4	15,718.4	28,841.0	15,107.9	70,354.6
gravitar	473.0	412.0	330.5	588.0	681.0	443.5	1,419.3
hero	20437.8	20130.2	27,153.9	20,818.2	33,860.9	5,053.1	55,887.4
ice_hockey	-1.9	-2.7	0.3	0.5	1.3	-2.1	1.1
kangaroo	7259.0	12992.0	14,492.0	14,854.0	12,909.0	12,117.0	14,637.5
krull	8422.3	7920.5	10,263.1	11,451.9	9,885.9	9,061.9	8,741.5
kung_fu_master	26059.0	29710.0	43,470.0	34,294.0	43,009.0	34,099.0	52,181.0
montezuma_revenge	0.0	0.0	0.0	0.0	367.0	0.0	384.0
ms_pacman	3085.6	2711.4	4,751.2	6,283.5	3,769.2	2,501.6	5,380.4
name_this_game	8207.8	10616.0	13,439.4	11,971.1	12,983.6	8,332.4	13,136.0
phoenix	8485.2	12252.5	32,808.3	23,092.2	34,775.0	16,974.3	108,528.6
pitfall	-286.1	-29.9	0.0	0.0	-2.1	-18.2	0.0
pong	19.5	20.9	20.7	21.0	20.8	21.0	20.9
private_eye	146.7	129.7	200.0	103.0	15,172.9	3,966.0	4,234.0
qbert	13117.3	15088.5	18,802.8	19,220.3	16,956.0	15,276.3	33,817.5
road_runner	39544.0	44127.0	62,785.0	69,524.0	63,366.0	41,681.0	62,041.0
robotank	63.9	65.1	58.6	65.3	54.2	53.5	61.4
sequest	5860.6	16452.7	44,417.4	50,254.2	4,754.4	2,495.4	15,898.9
skiing	-13062.3	-9021.8	-9,900.5	-8,857.4	-14,959.8	-16,307.3	-12,957.8
solaris	3482.8	3067.8	1,710.8	2,250.8	5,643.1	3,204.5	3,560.3
space_invaders	1692.3	2525.5	7,696.9	6,427.3	6,869.1	2,145.5	18,789.0
star_gunner	54282.0	60142.0	56,641.0	89,238.0	69,306.5	34,504.5	127,029.0
surround	-5.6	-2.9	2.1	4.4	6.2	-3.3	9.7
tennis	12.2	-22.8	0.0	5.1	23.6	0.0	-0.0
time_pilot	4870.0	8339.0	11,448.0	11,666.0	7,875.0	6,157.0	12,926.0
tutankham	68.1	218.4	87.2	211.4	249.4	231.6	241.0
venture	163.0	98.0	863.0	497.0	1,107.0	0.0	5.5
video_pinball	196760.4	309941.9	406,420.4	98,209.5	478,646.7	270,444.6	533,936.5
wizard_of_wor	2704.0	7492.0	10,373.0	7,855.0	15,994.5	5,432.0	17,862.5
yars_revenge	18089.9	11712.6	16,451.7	49,622.1	16,608.6	9,570.1	102,557.0
zaxxon	5363.0	10163.0	13,490.0	12,944.0	18,347.5	9,390.0	22,209.5

表 6: 无操作启动评估制度: 所有游戏的原始分数, 平均超过 200 个测试集, 来自训练期间获得最高分数的智能体快照。我们报告了 DQN、DDQN、Dueling DDQN 和 Prioritized DDQN 的已发布分数。对于分布式 DQN 和 Rainbow, 我们报告了我们自己对代理的评估。A3C 未列出, 因为该论文没有报告无操作制度的分数。

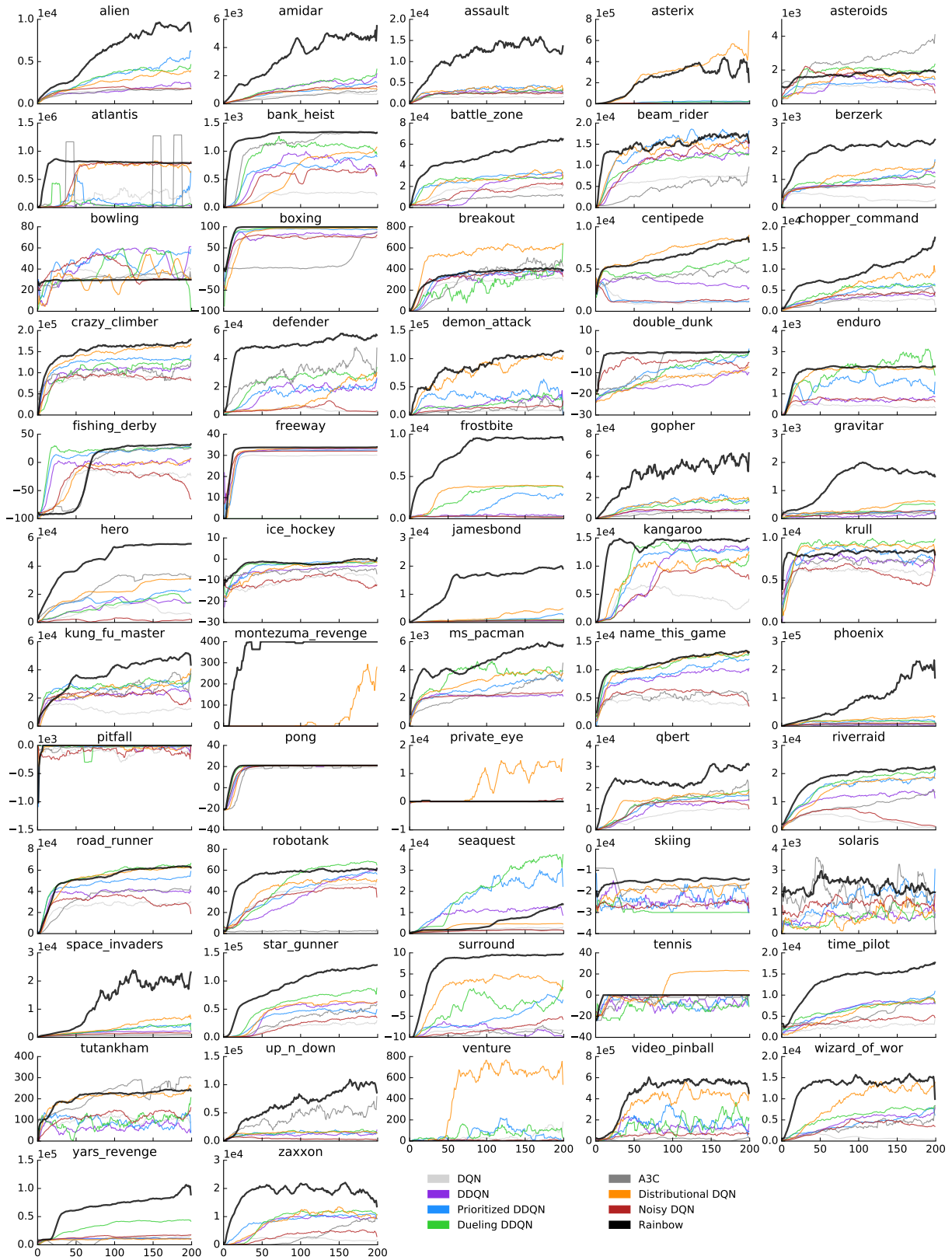


Figure 5: Learning curves for Rainbow and the baselines discussed in the paper, for each individual game. Every curve is smoothed with a moving average of 10 to improve readability.

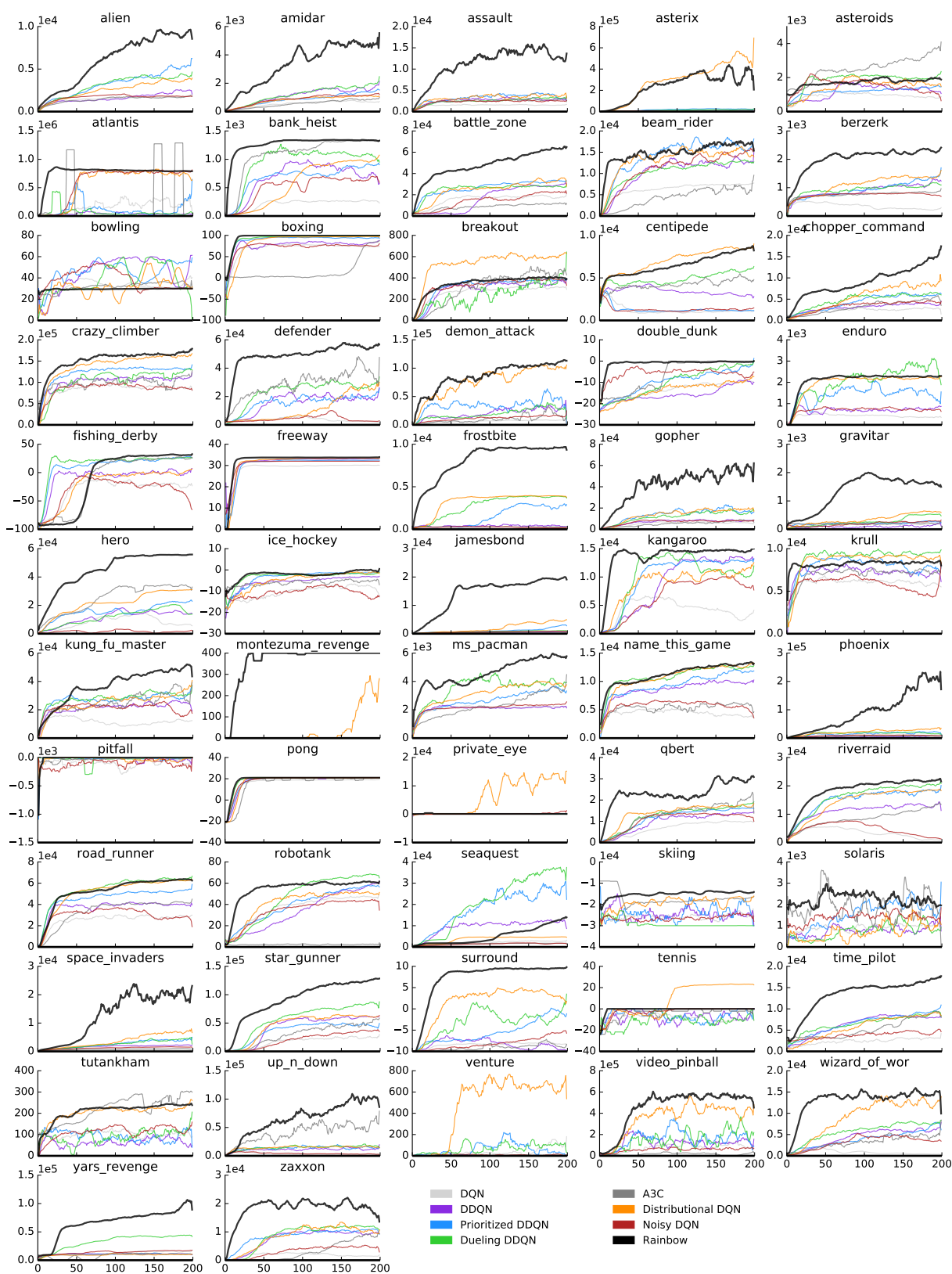


图 5: Rainbow 的学习曲线和本文中讨论的每个单独游戏的基线。每条曲线均使用 10 的移动平均值进行平滑, 以提高可读性。

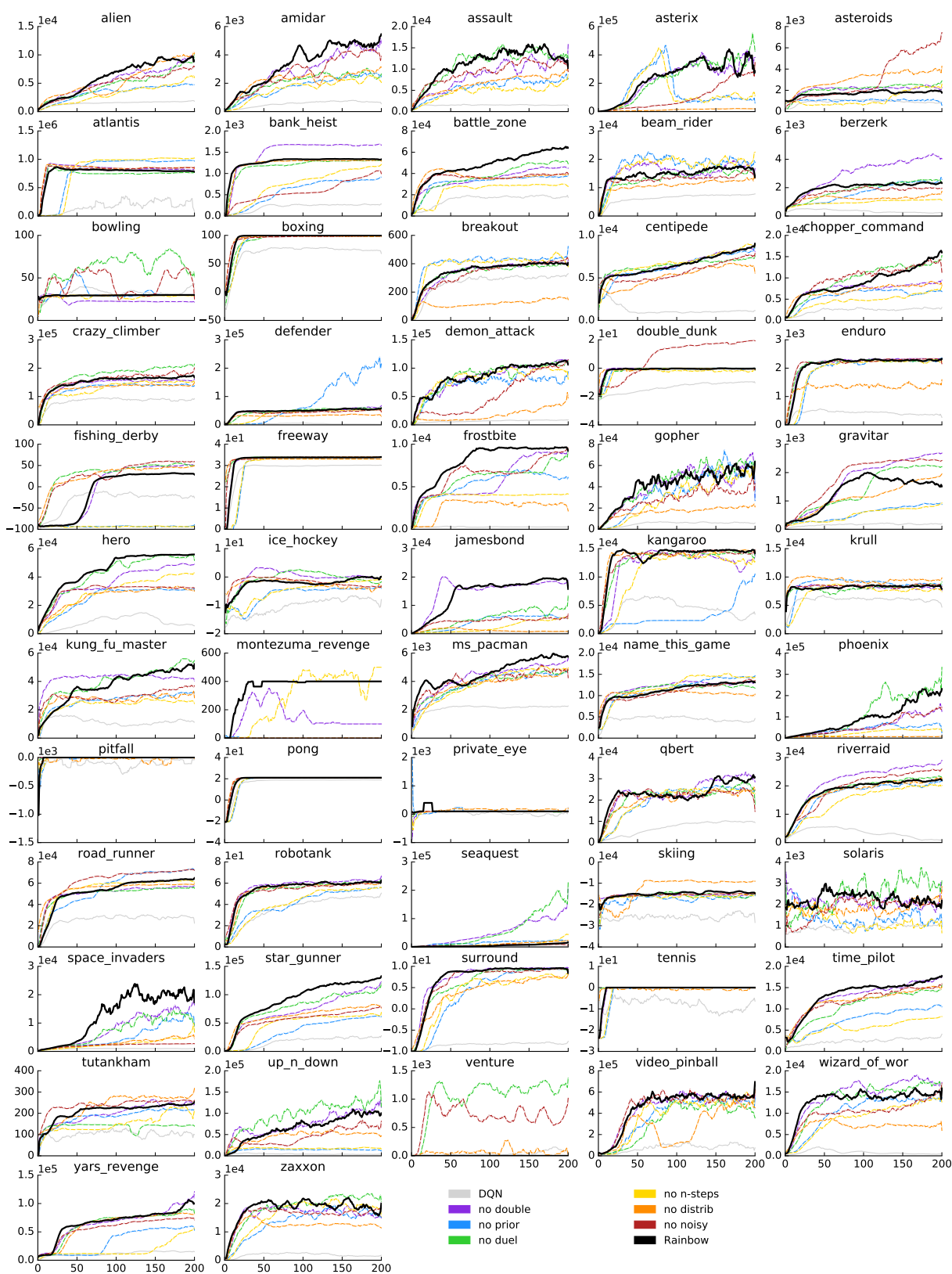


Figure 6: Learning curves for Rainbow and its ablations, for each individual game. Every curve is smoothed with a moving average of 10 to improve readability.

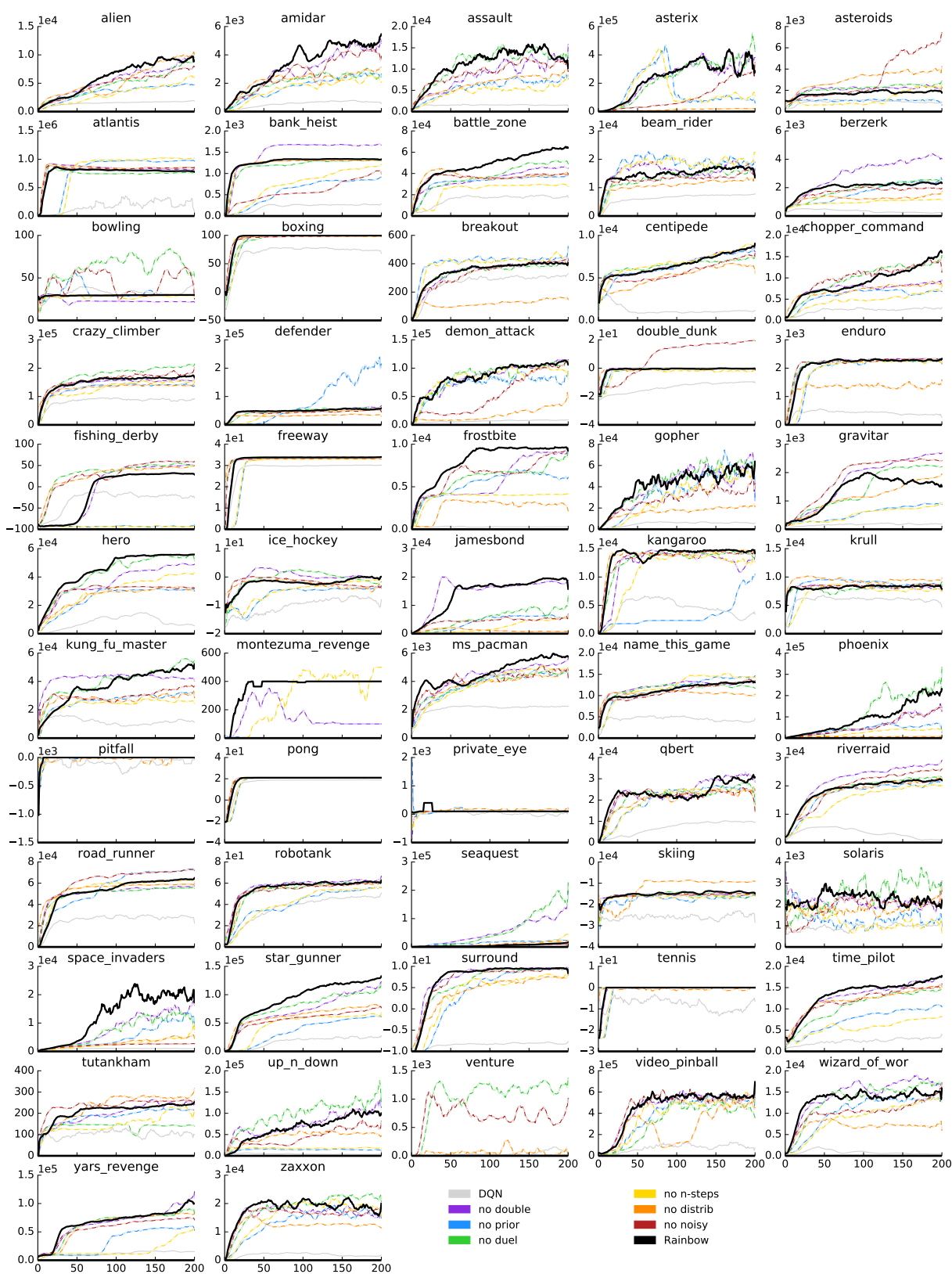


图 6: 每个单独游戏的 Rainbow 及其消融的学习曲线。每条曲线均使用 10 的移动平均值进行平滑, 以提高可读性。